Multiagent Systems

Study Text for the Final State Examination

Final State Examination in Artificial Intelligence, MFF UK

August 2026

This text covers the examination topic "Multiagent Systems" of the master's final state examination in Artificial Intelligence at the Faculty of Mathematics and Physics, Charles University. Its chapters correspond **one to one** to the sentences of the official wording of the topic (see the mapping table below). It is based on the slides of the lecture course NAIL106 *Multiagent Systems* (Neruda, Pilát, MFF UK) and on the textbooks M. Wooldridge: *An Introduction to MultiAgent Systems* (2nd ed., 2009), Y. Shoham, K. Leyton-Brown: *Multiagent Systems* (2009) and S. Russell, P. Norvig: *Artificial Intelligence: A Modern Approach*. This is an English translation of the Czech study text *Multiagentní systémy*; terminology follows the standard usage of the sources listed above.

Scope and marking

The length of the chapters reflects the weight of the material: most space goes to agent architectures, communication, and cooperation with game theory — that is, to what the lecture treats in most detail.

The marker [♦ beyond the slides] flags material that is **not in the slides** of the lecture but has been kept in the text — either because *the official wording of the topic names it*, or because the surrounding exposition would not make sense without it. It is an offer for deletion: whatever is not marked is backed by the slides.

Chapters 3 (pathfinding) and 6 (agent learning) are not in the slides at all — the lecture explicitly lists them among the topics it does not cover — but the examination topic enumerates them, so they have to be known. They are therefore treated briefly, at a level that will do at the examination.

Mapping of the examination topic to chapters

Sentence of the official topic	Chapter in this text
The architecture of an autonomous agent; the action selection mechanism.	1 Architecture of an autonomous agent and the action selection mechanism
Methods for controlling agents; symbolic and connectionist reactive planning, hybrid approaches.	2 Methods for controlling agents
The pathfinding problem.	3 The pathfinding problem
Communication and knowledge in multiagent systems, ontologies, speech acts, FIPA-ACL, protocols.	4 Communication and knowledge in MAS
Distributed problem solving, cooperation, Nash equilibria, Pareto efficiency, resource allocation, auctions.	5 Distributed problem solving, cooperation, game theory, and auctions
Methods for agent learning; reinforcement learning.	6 Agent learning and reinforcement learning
Design methodologies, languages, and environments of multiagent systems.	7 Design methodologies, languages, and MAS platforms

Contents

1 Architecture of an autonomous agent and the action selection mechanism

1.1 The agent and the multiagent system

Multiagent systems (MAS) emerged as an independent area of computer science in the 1990s in response to several concurrent trends: the *ubiquity* of computing technology, *connectivity and distribution* (computation as interaction), the growing *complexity* of tasks, the *delegation* of tasks to software, and the pressure towards ease of use. The classical paradigm "a program receives an input, computes an output, and terminates" ceases to be adequate the moment software is permanently embedded in an environment that changes independently of it and in which other independent components run alongside it.

Agent — working definitions

Russell, Norvig (AIMA): An agent is anything that can be viewed as an entity *perceiving* its environment through sensors and *acting* upon that environment through effectors.
Pattie Maes: Autonomous agents are computational systems that inhabit some complex dynamic environment, sense and act autonomously in this environment, and by doing so realise a set of goals or tasks for which they were designed.
Wooldridge, Jennings: An agent is a computer system situated in some environment that is capable of *autonomous action* in that environment in order to meet its delegated goals.

There is no single agreed definition of an agent — Franklin and Graesser (1996), in the paper *Is it an agent, or just a program?*, collected dozens of mutually incompatible definitions. The crucial point is to distinguish an agent from an ordinary procedure or object. The most frequently cited formulations:

Multiagent system

A multiagent system is a system composed of several agents that *interact* with one another — most often by exchanging messages over a computer network. The agents *need not share a common goal*; they may have their own, even conflicting, interests. This is why it is necessary to study mechanisms of negotiation, coordination, and dynamic resource allocation.

Autonomy. A human being is fully autonomous, a Java method not at all. An agent lies in between: it should autonomously choose the *way* in which a given problem is to be solved (that is, the choice of subgoals), but not the main goal — that is delegated to it.

Delimitation against related fields. The breadth of the topic is explained by the fact that MAS can be read from five different sides:

- **Software engineering.** An agent is the next level of abstraction in the sequence machine code → structured programs → objects → distributed objects (CORBA) → agents. The classic quip: "objects do it for free, agents do it because they want to".
- **Distributed and parallel systems.** Decades of research have solved mutual exclusion, deadlock, and consensus. The difference is that agents are *autonomous*, coordination mechanisms are not given in advance, and everything has to be resolved at run time.
- **Artificial intelligence.** Traditionally: MAS is a subfield of AI. According to Russell and Norvig the goal of AI is conversely the design of intelligent agents (hence AI $\subseteq$ MAS); Etzioni's pointed version is "MAS is 1 % AI and 99 % computer science". MAS uses AI techniques (knowledge representation, planning) but emphasises the *social* aspects that classical AI overlooked.
- **Game theory and economics.** A formal apparatus for rational decision making by multiple parties. Mathematical game theory deals mainly with questions of *existence*, whereas MAS deals with the *computational* aspects (complexity, practical usability).

- **Sociology.** A key notion of MAS is *agent societies*, and sociology studies human societies — so the inspiration suggests itself. The same caveat applies here as to the relation between AI and human intelligence: inspiration yes, necessity no (aeroplanes do not flap their wings). In the opposite direction the link is stronger: sociology, economics, and ecology all use agents as a building block of their simulation models — *agent-based modelling* (ABM), see the platforms in chapter 7.

1.2 Properties of an intelligent agent

- **Reactivity** — it perceives the environment and is able to respond to changes in it within a reasonable (real) time.
- **Proactiveness** — it has its own goals and actively pursues them, on its own initiative, not merely as a response to a stimulus.
- **Social ability** — it is able to communicate with other agents and with humans.

Achieving *purely* reactive or *purely* goal-oriented behaviour is fairly easy. What is hard is to find the **balance between reactivity and proactiveness**: an agent must not blindly follow a plan that has become pointless, but neither may it constantly reconsider its goals and never get anywhere. This trade-off recurs in various guises throughout the whole text (see in particular agent commitment, section 2.5, and hybrid architectures, section 2.11).

1.3 Intentional systems and the intentional stance

When we talk about agents, we ascribe *mental states* to them: "the agent believes that…", "the agent wants…". Daniel Dennett defines an **intentional system** as an entity whose behaviour can be predicted by ascribing to it properties such as beliefs, desires, and rationality. Intentional systems form a hierarchy: a *first-order* system has beliefs and desires, a *second-order* system has beliefs and desires about beliefs and desires (its own as well as those of others), and so on.

Dennett distinguishes three *stances* towards predicting the behaviour of a system: the *physical* one (the laws of physics suffice — if I throw a stone I need not speak of its desires, mass, velocity, and gravity are enough), the *design stance* (I do not know all the physical laws, but I know the purpose for which the system was designed — I can set an alarm clock without understanding its electronics), and the *intentional* one (I describe the system in terms of beliefs, desires, and rationality).

Shoham gives a provocative example: "The light switch in the room is a very cooperative agent that is willing of its own free will to conduct electric current, but does so only when it believes that we really want it to." The description is consistent and elegant, yet most people find it absurd — *because for a light switch we have a simpler and equally accurate description.* The intentional stance pays off where a physical or design description does not exist or is impractical (even with the schematic of a computer at hand it is hard to derive why clicking an icon opens a window). **From the point of view of computer science, the intentional stance is above all an abstraction for managing complexity** — and for many programmers that is precisely the main contribution of MAS. Dennett himself adds a philosophical question: in his view intentionality does not exist in itself, it arises only *in the process of interpretation* by an observer.

1.4 The environment and its properties

An agent's behaviour is fundamentally determined by the type of environment it finds itself in. The standard classification (Russell, Norvig):

Dichotomy	Meaning
fully / partially observable	The agent's sensors give it the complete state of the environment — or only part of it (hidden information, noise, limited range).
static / dynamic	The environment changes only as a result of the agent's actions — or also otherwise (other agents, natural processes). In a static environment the agent can "think its next move over at leisure".
deterministic / non-deterministic	Every action has exactly one possible outcome — or several (with a known or unknown probability distribution; in that case we speak of a stochastic environment).
discrete / continuous	A finite number of states and actions — or continuous quantities (position, velocity).

Further commonly listed dichotomies: *episodic / sequential* (does today's action affect future rewards?), *single-agent / multiagent*, *known / unknown* (does the agent know the transition function in advance?).

An unpleasant observation: most interesting environments (the real world, the internet) are continuous, non-deterministic, dynamic, and only partially observable. Precisely for this reason it is not enough to program an agent as a fixed table of rules.

Example — a thermostat. A not particularly intelligent agent, situated in an environment (a room). Two percepts ("it is cold", "the temperature is fine"), two actions (switch on, switch off), and a decision unit: cold ⇒ switch on, OK ⇒ switch off. The environment is nevertheless non-deterministic (someone opens the door).

Example — a software daemon. For instance the Unix `xbiff`: the environment is the operating system, it perceives by calling system services, and its actions are software operations (start a mail client, change an icon). Its decision algorithm is at the level of the thermostat.

1.5 An abstract agent architecture

We now formalise the view of an agent and its interaction with the environment. We assume the environment is discrete — which is not restrictive, as every continuous environment can be discretised to arbitrary precision.

Environment, agent, run

The **environment** can be in one of finitely many discrete states, $E = \{e, e', \dots\}$. The **agent** has a finite set of actions at its disposal, $A = \{a, a', \dots\}$.
A **run** of an agent in an environment is a sequence of alternating environment states and actions:

$$r = e_0 \xrightarrow{a_0} e_1 \xrightarrow{a_1} e_2 \xrightarrow{a_2} \cdots \xrightarrow{a_{n-1}} e_n.$$

Let R denote the set of all finite runs (over E and A), $R^A \subseteq R$ those that end with an action, and $R^E \subseteq R$ those that end with an environment state.

State transformer function of the environment

$$\tau : R^A \to 2^E$$

The function τ maps a run ending in an action to the set of states the environment may reach after that action. Two consequences follow from this definition:

- **The environment has a history** — the next state depends not only on the last action but on the entire history of the run.
- **The environment is non-deterministic** — the result is a set of states, and we do not know in advance which one will occur.

If $\tau(r) = \emptyset$, the run has ended. In what follows we assume that all runs terminate.
An **environment** is a triple $Env = \langle E, e_0, \tau \rangle$, where e_0 is the initial state.

Agent and system

An **agent** is a function mapping runs that end in an environment state to actions:

$$Ag : R^E \to A.$$

An agent therefore decides on the basis of the *entire history* of the system, and it decides *deterministically*. Let $\mathcal{AG}$ denote the set of all agents.
A **system** is a pair $\langle Ag, Env \rangle$. The sequence $(e_0, a_0, e_1, a_1, \dots)$ is a *run of agent Ag in environment Env* if and only if
1. e_0 is the initial state of Env,
2. $a_0 = Ag(e_0)$,
3. for every $u > 0$ we have $e_u \in \tau\big((e_0, a_0, \dots, a_{u-1})\big)$ and $a_u = Ag\big((e_0, a_0, \dots, e_u)\big)$.

The set of all runs of agent Ag in Env is denoted $R(Ag, Env)$.

Functional equivalence of agents

Agents Ag_1 and Ag_2 are **functionally equivalent with respect to** Env if and only if $R(Ag_1, Env) = R(Ag_2, Env)$.
They are **simply functionally equivalent** if they are functionally equivalent with respect to *all* environments.

Functional equivalence matters as a tool for comparing the *expressive power* of architectures: it says that the internal implementation is irrelevant as long as it leads to the same observable runs.

1.6 Purely reactive agents and agents with internal state

Purely reactive (tropistic) agent

$$Ag : E \to A$$

It reacts only to the current state of the environment and ignores the history.
Relation to the general agent: for every purely reactive agent there exists a functionally equivalent standard agent; the converse does *not* hold.
Thermostat: $Ag(e) =$ off for $e =$ "temperature OK", otherwise $Ag(e) =$ on.

Agent with internal state

Let I be the set of internal states of the agent and Per the set of its percepts. The agent is determined by a triple of functions:

$$\begin{aligned} see &: E \to Per && \text{(perception)} \\ next &: I \times Per \to I && \text{(internal state update)} \\ action &: I \to A && \text{(action selection)} \end{aligned}$$

The cycle: the agent starts in state i_0; it perceives the environment ($see(e)$); it updates its state ($i \leftarrow next(i, see(e))$); it selects an action ($a \leftarrow action(i)$); it executes the action and repeats.
Theorem (without proof): For every agent with internal state there exists a functionally equivalent standard agent. Internal state therefore does *not increase expressive power*, but it is essential for the *efficiency* and clarity of the implementation.

The function *see* at the same time formalises **observability**: if *see* maps two different states $e_1 \neq e_2$ to the same percept, the agent cannot distinguish them — the environment is only partially

observable for it. The extreme cases are $|Per| = |E|$ with *see* injective (fully observable) versus $|Per| = 1$ (the agent sees nothing).

1.7 How to tell an agent what to do: utility

We do not want agents with hard-wired behaviour. We want to tell the agent *what* to do, not *how*. The standard AI approach is to define the goal *indirectly*, by a measure of success.

Utility

Utility of a state: $u : E \to \mathbb{R}$. The agent's goal is to reach states with a high utility value. Evaluating individual states is short-sighted. We therefore introduce the **utility of a run**:

$$u : R \to \mathbb{R}.$$

This corresponds better to agents that run over the long term. The utility of a run can be derived from states, e.g. as the utility of the *worst* state visited or as the *average* utility of the states visited — which is better depends on the task.

Optimal agent

Let $P(r \mid Ag, Env)$ be the probability of run r of agent Ag in environment Env, where $\sum_{r \in R(Ag,Env)} P(r \mid Ag, Env) = 1$. An optimal agent maximises *expected utility*:

$$Ag_{\text{opt}} = \arg\max_{Ag \in \mathcal{AG}} \sum_{r \in R(Ag,Env)} u(r)\, P(r \mid Ag, Env).$$

The definition is elegant but **non-constructive**: it does not say how to design such an agent. Moreover, it is often difficult to specify the utility function at all. (A note: money is not a good utility for a human being — the utility of money is non-linear and differs for the rich and the poor.)

Example: Tileworld. A classical testbed for agent architectures. Agents move around a chessboard in four directions; the board contains holes, obstacles, and tiles, and the goal is to push the tiles into the holes. The environment is *dynamic* — holes, obstacles, and tiles appear and disappear at random. The utility of a run is

$$u(r) = \frac{N_f}{N_a},$$

where N_f is the number of holes filled by the agent during run r and N_a the number of all holes that appeared during r. The agent has to be able both to *react* to changes (the tile I am pushing has disappeared) and to *exploit opportunities* (a new tile has appeared next to me). Historically, Tileworld was used precisely to investigate the trade-off between reactivity and proactiveness (see agent commitment, section 2.5).

1.8 Predicate task specification

A special case of utility is a mapping into $\{0, 1\}$: a run is either successful ($u(r) = 1$) or not.

Task environment

Let $\Psi : R \to \{0, 1\}$ be a **predicate specification**: $\Psi(r)$ holds if and only if $u(r) = 1$. A **task environment** is a pair $\langle Env, \Psi \rangle$; it specifies both the properties of the system (through Env) and the criterion for completing the task (through Ψ).
Let $R_\Psi(Ag, Env) = \{r \in R(Ag, Env) \mid \Psi(r)\}$.

When does an agent *solve* a task? There are three readings:

- **Pessimistic:** $R_\Psi(Ag, Env) = R(Ag, Env)$ — *all* runs satisfy Ψ.

- **Optimistic:** $\exists r \in R(Ag, Env) : \Psi(r)$ — *at least one* run.
- **Realistic:** we extend τ with a probability distribution and measure success by the probability $P(\Psi \mid Ag, Env) = \sum_{r \in R_\Psi(Ag,Env)} P(r \mid Ag, Env)$.

Types of tasks

Achievement tasks — "find something". A set of goal states $G \subseteq E$ is given; $\Psi(r)$ holds if at least one state from G occurs in run r. Typically every classical AI task (search).
Maintenance tasks — "avoid something". A set of "bad" states $B \subseteq E$ is given; $\Psi(r)$ fails to hold if a state from B occurs in r. Typically games, where B are the losing positions and the environment is the opponent.
Combination: reach a state from G while avoiding the states in B.

1.9 The action selection mechanism — an overview

Action selection mechanism

The procedure by which an agent decides at each moment which of the available actions to perform, on the basis of percepts from the environment and its own internal state. An **agent architecture** is a software architecture that realises such a mechanism — according to Maes (1991) "a particular methodology for building agents that specifies how the agent can be decomposed into a set of components and how these components should interact so that together they answer the question of how the sensor data and the current internal state determine the actions and the future internal state of the agent."

Action selection is the **key problem of agent design**, and the whole of chapter 2 is nothing but a survey of how it can be realised:

Family	Action selection mechanism	Typical representative
symbolic / deliberative	logical deduction over a database of formulae; planning	deductive agent, STRIPS, BDI, PRS
reactive (symbolic)	a set of "situation → action" rules with fixed priorities	subsumption architecture (Brooks)
reactive (connectionist)	activation spreading in a network of modules, the most active one wins	agent network architecture (Maes)
hybrid	layers of different abstraction + arbitration between them	TouringMachines, InteRRaP
coalitional	competition of dynamically arising coalitions of processes for "consciousness"	IDA / global workspace
learned	maximisation of a learned Q-function / stochastic policy	Q-learning, actor-critic (chapter 6)

1.10 Exam summary

- Agent = an autonomous system situated in an environment, which perceives and acts in it in order to meet delegated goals. MAS = a system of interacting agents *without a necessarily shared goal*.
- Three properties of an intelligent agent: reactivity, proactiveness, social ability; the hard part is balancing them.
- Dennett: the intentional system and the three stances (physical, design, intentional); the intentional stance is an abstraction for managing complexity.
- The environment: 4 dichotomies (observability, static/dynamic, determinism, discreteness); real environments are of the hardest kind.
- The formalism: E, A, a run $r = e_0 a_0 e_1 \ldots e_n$, R^A/R^E, the state transformer function $\tau : R^A \to 2^E$ (hence history + non-determinism), $Env = \langle E, e_0, \tau \rangle$, an agent $Ag : R^E \to A$, a system

$\langle Ag, Env \rangle$.

- Functional equivalence (with respect to an environment / simple). A purely reactive agent $Ag : E \to A$ (weaker); an agent with internal state (*see*, *next*, *action*) — both reducible to a standard agent.
- Utility: $u : E \to \mathbb{R}$ vs. $u : R \to \mathbb{R}$; the optimal agent maximises *expected* utility — the definition is non-constructive. Tileworld and $u(r) = N_f/N_a$.
- The task environment $\langle Env, \Psi \rangle$; pessimist/optimist/realist; achievement (G) and maintenance (B) tasks.
- The **action selection mechanism** is the core of the question: be able to list the families of mechanisms and their representatives (the table above).

2 Methods for controlling agents: symbolic and connectionist reactive planning, hybrid approaches

This chapter is the core of the topic: it goes through the individual *methods of controlling an agent*, that is, the concrete realisations of the action selection mechanism from section 1.9. It proceeds from the symbolic (deliberative) pole to the reactive one and ends with the hybrid architectures that join the two:

Approach	Where it is in this chapter
symbolic (deliberative) control	the deductive agent, BDI, STRIPS, PRS (sections 2.1–2.6)
symbolic reactive planning	the subsumption architecture (Brooks), section 2.8
connectionist reactive planning	the agent network architecture (Maes), section 2.9
hybrid approaches	layered architectures, TouringMachines, InteRRaP, IDA, section 2.11 onwards

2.1 The classical AI approach and the deductive agent

The classical way in which AI builds an "intelligent system" rests on two pillars:

1. **Symbolic representation** of the environment and of behaviour — here specifically by logical formulae.
2. **Syntactic manipulation** of this representation — that is, logical deduction, theorem proving.

The theory of the agent's behaviour is then also its program — an *executable specification* that generates concrete actions. This approach runs into two classical problems:

- **The transduction problem**: how to translate the real world into a symbolic representation (computer vision, natural language processing, learning). In Goethe's words: "All theory, dear friend, is grey, but the golden tree of life springs ever green."
- **The representation and reasoning problem**: how to represent knowledge and how to reason over it efficiently (theorem provers, planners).

A digression: deduction vs. induction. *Deduction* is the derivation of conclusions that are true whenever the premises are true — a move from the general to the particular (the syllogism: "All men are mortal, Socrates is a man, therefore Socrates is mortal"). *Induction* means that, given true premises, the conclusion is more likely true than false — from the particular to the general. Sherlock Holmes in fact uses induction, even though he calls it deduction; mathematical induction, on the contrary, is deduction.

Deductive agent (the agent as a theorem prover)

Let L be a set of first-order predicate logic formulae and $D = 2^L$ the set of databases of such formulae. The agent's internal state is a database $DB \in D$. Reasoning is realised by inference rules ρ:

$$DB \vdash_\rho \varphi \quad \text{— formula } \varphi \text{ is derivable from } DB \text{ using the rules } \rho.$$

The agent is then a triple of functions $see : E \to Per$, $next : D \times Per \to D$, $action : D \to A$, where *action* is defined by the procedure below.

Action selection as theorem proving.

```
1:  function ACTION(DB : D) : A
2:      for all a ∈ A do
3:          if DB ⊢_ρ Do(a) then return a
4:          end if
5:      end for
6:      for all a ∈ A do
7:          if DB ⊬_ρ ¬Do(a) then return a
8:          end if
9:      end for
10:     return null
11: end function
```

The first loop looks for an action that can be *proved* to be desirable ($Do(a)$ is derived). If there is no such action, the second loop looks for an action that is at least *consistent* with the database (it cannot be proved that it should not be done). If that fails too, the agent does nothing.

Example: the 3×3 vacuum world. Predicates: $In(x, y)$ (the agent is at position (x, y)), $Dirt(x, y)$ (there is dirt at (x, y)), $Facing(d)$ (the agent faces direction d). Rules for deriving an action:

$$\begin{aligned} In(x, y) \wedge Dirt(x, y) &\Rightarrow Do(suck) \\ In(0, 0) \wedge Facing(north) \wedge \neg Dirt(0, 0) &\Rightarrow Do(forward) \\ In(0, 2) \wedge Facing(north) \wedge \neg Dirt(0, 2) &\Rightarrow Do(turn) \qquad \ldots \end{aligned}$$

The agent thus systematically traverses the world in a "snake" pattern 00–01–02–12–11–10–... Note that cleaning has higher priority than movement (the rule for *suck* is listed first and the other rules have the negation of *Dirt* in their premises).

Pros and cons.

+ Elegant, with clear (and beautiful) semantics; specification = program.
− Practically unusable for non-trivial tasks: proving is slow and need not terminate.
− **The problem of calculative rationality**: the agent derives the optimal action for the state of the environment as it was *at the beginning* of the derivation — meanwhile the environment has changed and the action is no longer optimal. A disaster in a rapidly changing environment.
− The function *see* is hard to design (how to turn an image into formulae, how to represent time).

Nevertheless, individual elements of this approach reappear in later architectures, particularly in BDI.

2.2 Practical reasoning: the BDI architecture

Practical reasoning

Inspired by human decision making. *Theoretical* reasoning (Socrates) leads only to what we think about the world; *practical* reasoning leads to **actions**. It has two phases:

- **Deliberation** — *what we want to achieve* ("I want to finish my master's degree").
- **Means-ends reasoning**, that is planning — *how we are going to achieve it* (construct a plan for finishing the degree).

And neither may take too long.

Beliefs — Desires — Intentions

Beliefs — the informational state of the agent, its knowledge about the world. In MAS we deliberately do not call them "knowledge", in order to stress that they are *subjective*, *need not be true*, and *may change in the future*. They may also contain inference rules that permit forward chaining.

Desires — the motivational state of the agent, the goals or situations it would like to achieve. Desires *may be mutually inconsistent* (I want both to study and to go for a beer).

Intentions — states of the world that the agent has *committed* itself to achieving. Intentions lead to actions: it is for their sake that the agent plans. Intentions **persist** until

1. the agent achieves them, or
2. it comes to believe that they are unachievable, or
3. the reasons that led to them cease to hold.

Moreover, intentions *constrain further deliberation* (I will not reconsider options incompatible with an intention I have adopted) and *influence future beliefs* (I count on my intention succeeding).

The difference between a desire and an intention is illustrated by Bratman (1990): "My desire to play basketball this afternoon is merely a potential influencer of my conduct this afternoon. It must vie with my other relevant desires... In contrast, once I *intend* to play basketball this afternoon, the matter is settled: I normally need not continue to weigh the pros and cons; when the afternoon arrives, I will normally just proceed to execute my intention."

The deliberation functions

Let $B \subseteq Bel$ denote the agent's current beliefs, $D \subseteq Des$ its current desires, and $I \subseteq Int$ its current intentions. Deliberation consists of three functions:

- $brf : 2^{Bel} \times Per \to 2^{Bel}$ — the *belief revision function*, updating beliefs according to a percept;
- $options : 2^{Bel} \times 2^{Int} \to 2^{Des}$ — generating options (desires) from beliefs and existing intentions;
- $filter : 2^{Bel} \times 2^{Des} \times 2^{Int} \to 2^{Int}$ — selecting intentions from the options.

Splitting deliberation into *option generation* and *filtering* is essential: generation is cheap and broad, whereas filtering is the actual decision making (it also takes existing commitments into account).

2.3 Planning: STRIPS

Means-ends reasoning / planning

The process of deciding how to achieve a goal (an intention) using the available means (actions).

- **Input:** the goal (= intention I); the current state of the environment (= beliefs B); the actions available to the agent (Ac).
- **Output:** a plan = a sequence of actions after whose execution the goal is satisfied.

STRIPS (Fikes, Nilsson, 1971) models the world by a set of first-order logic formulae. Each action

is described by a *schema* with preconditions and effects (facts added and deleted). The planning algorithm finds the difference between the goal and the current state and reduces it by a suitable action (*means-ends analysis*). This is elegant but not very practical — the algorithm often gets bogged down in low-level details.

Action descriptor, planning problem, plan

An **action descriptor** a is a triple $\langle P_a, D_a, A_a\rangle$: P_a — preconditions, D_a — the delete list, A_a — the add list; for simplicity these are ground atomic formulae (without connectives and variables).
A **planning problem** is a triple $\langle B_0, O, G\rangle$, where B_0 are the initial beliefs, $O = \{\langle P_a, D_a, A_a\rangle : a \in Ac\}$ the descriptors of all actions, and G a set of formulae describing the goal.
A **plan** $\pi = (a_1, \ldots, a_n)$, $a_i \in Ac$, determines a sequence of databases

$$B_i = (B_{i-1} \setminus D_{a_i}) \cup A_{a_i}, \qquad i = 1, \ldots, n.$$

A plan is **admissible** if $B_{i-1} \models P_{a_i}$ for every i. A plan is **correct (valid)** if it is admissible and moreover $B_n \models G$.
Planning means, for a given $\langle B_0, O, G\rangle$, finding a correct plan or declaring that none exists.

Example: the blocks world. Predicates: $On(x, y)$ (x is on y), $OnTable(x)$, $Clear(x)$ (nothing is on x), $Holding(x)$ (the arm is holding x), $ArmEmpty$. Actions:

Action	Pre / Del	Add
$Stack(x, y)$	Pre: $\{Clear(y), Holding(x)\}$ Del: $\{Clear(y), Holding(x)\}$	$\{ArmEmpty, On(x, y)\}$
$UnStack(x, y)$	Pre: $\{On(x, y),\ Clear(x), ArmEmpty\}$ Del: $\{On(x, y), ArmEmpty\}$	$\{Holding(x), Clear(y)\}$
$Pickup(x)$	Pre: $\{OnTable(x),\ Clear(x), ArmEmpty\}$ Del: $\{OnTable(x), ArmEmpty\}$	$\{Holding(x)\}$
$PutDown(x)$	Pre: $\{Holding(x)\}$ Del: $\{Holding(x)\}$	$\{ArmEmpty, OnTable(x)\}$

Initial state $B_0 = \{Clear(A), On(A, B), OnTable(B), OnTable(C), Clear(C), ArmEmpty\}$, goal $G = \{OnTable(A), OnTable(B), OnTable(C)\}$.

A correct plan: $\pi = (UnStack(A, B),\ PutDown(A))$. Verification:

- $B_0 \models \{On(A, B), Clear(A), ArmEmpty\}$, so the first action is admissible; after it

$$B_1 = \big(B_0 \setminus \{On(A, B), ArmEmpty\}\big) \cup \{Holding(A), Clear(B)\}.$$

- $B_1 \models \{Holding(A)\}$, so the second action is admissible too; after it

$$B_2 = \big(B_1 \setminus \{Holding(A)\}\big) \cup \{ArmEmpty, OnTable(A)\}.$$

- $B_2 \models G$, so the plan is correct.

Auxiliary functions over plans

Plan — the set of all plans over Ac; $pre(\pi)$ — the precondition of a plan; $body(\pi)$ — the body of a plan (the sequence of actions); $empty(\pi)$ — is the plan empty?; $execute(\pi)$ — executes all actions of the plan; $hd(\pi)$ — the first action; $tail(\pi)$ — the rest; $sound(\pi, I, B)$ — plan π is correct with respect to intentions I and beliefs B.
The agent's planning function: $plan : 2^{Bel} \times 2^{Int} \times 2^{Ac} \to Plan$.

The plan library. The agent need not construct plans online — that is expensive. Very often $plan()$ is implemented as a *library of ready-made plans*: it suffices to traverse it once and check

whether the preconditions of a plan match the current beliefs and whether the effects of the plan match the goal. This idea is the core of the PRS architecture (section 2.6).

2.4 Implementing a BDI agent

The main control loop of a BDI agent:

1: $B \leftarrow B_0$; $\quad I \leftarrow I_0$
2: **while true do**
3: $\quad v \leftarrow see()$
4: $\quad B \leftarrow brf(B, v)$
5: $\quad D \leftarrow options(B, I)$
6: $\quad I \leftarrow filter(B, D, I)$
7: $\quad \pi \leftarrow plan(B, I, Ac)$
8: $\quad$ **while** $\neg\big(empty(\pi) \vee succeeded(I, B) \vee impossible(I, B)\big)$ **do**
9: $\quad\quad a \leftarrow hd(\pi)$; $\quad execute(a)$; $\quad \pi \leftarrow tail(\pi)$
10: $\quad\quad v \leftarrow see()$; $\quad B \leftarrow brf(B, v)$
11: $\quad\quad$ **if** $reconsider(I, B)$ **then**
12: $\quad\quad\quad D \leftarrow options(B, I)$; $\quad I \leftarrow filter(B, D, I)$
13: $\quad\quad$ **end if**
14: $\quad\quad$ **if** $\neg\, sound(\pi, I, B)$ **then**
15: $\quad\quad\quad \pi \leftarrow plan(B, I, Ac)$ ▷ replanning
16: $\quad\quad$ **end if**
17: $\quad$ **end while**
18: **end while**

The meaning of the predicates: $succeeded(I, B)$ — the intentions I hold given the beliefs B; $impossible(I, B)$ — the intentions I cannot hold given B. The inner loop therefore terminates when the plan is exhausted, the goal is achieved, or the goal has become unachievable.

2.5 Agent commitment

Commitment strategies towards intentions

When should an agent drop a goal?

- **Blind commitment** (also *fanatical*) — the agent maintains an intention until it *comes to believe that it has achieved it.* (It never gives up.)
- **Single-minded commitment** — it maintains an intention until it comes to believe that it has achieved it *or* that it is no longer achievable.
- **Open-minded commitment** — an intention persists as long as the agent believes it is still achievable (and still desired) — so it may also be dropped when the motivation changes.

Commitment to a plan versus **commitment to a goal**. The agent in the algorithm above is committed to a single plan; it abandons it when it believes the goal is satisfied, that it is unachievable, or when the plan is empty. But the harder question remains: when should the agent *reconsider the intention itself*?

The classical dilemma: deliberation **takes time** and the environment changes while it is going on.

- An agent that *does not reconsider* its intentions may cling to goals that can no longer be achieved.
- An agent that reconsiders *too often* is so busy reasoning that it gets nothing done.

Meta-level control (Wooldridge, Parsons, 1999): a function *reconsider*() that is *computationally cheaper* than the reconsideration itself. The quality criterion: *reconsider*() works well if, whenever it proposes reconsideration, the agent *actually* changes its intention after deliberating (and vice versa).

Bold vs. cautious agent

A **bold agent** — reconsiders its intentions only after completing the entire plan (it never interrupts a plan for the sake of deliberation).
A **cautious agent** — reconsiders after every action of the plan.
The **level of boldness** — how many actions the agent performs between two deliberations.
The **rate of world change** — how many times the environment changes during one cycle of the agent.
Agent effectiveness = the number of intentions achieved / the number of all intentions.

Experimental result (Kinny, Georgeff, in Tileworld):

- If the world changes *slowly*, **bold** agents are more effective — deliberation would be needless overhead.
- If the world changes *rapidly*, **cautious** agents outperform them — plans become obsolete quickly.

This is a concrete quantitative form of the general reactivity vs. proactiveness trade-off.

2.6 The Procedural Reasoning System (PRS)

PRS — *Procedural Reasoning System*

Georgeff et al., 1980s, Stanford. **The first implementation of the BDI architecture** and one of the most successful agent architectures in practice — it has been reimplemented many times: **AgentSpeak/Jason**, **JAM**, **JACK**, **Jadex**.
Practical deployments: OASIS (the air traffic control system in Sydney), SPOC (business process organisation), SWARMM (an air combat simulator of the Australian air force).

Components of a PRS agent: a database of *beliefs* (FOL atoms of the Prolog kind), *desires* (goals), a *plan library*, a stack of *intentions*, and an *interpreter* that ties everything together. Sensor data enter the beliefs, and the interpreter issues actions.

A plan in PRS

The agent has a library of ready-made plans representing its *procedural knowledge*. **Nothing is planned from scratch**; plans are only *selected* from the library. A plan has three parts:

- **Goal** (also the *invocation condition* / *cue*) — the condition that is to hold after the plan has been executed; it determines which goal the plan is suited to.
- **Context** — the condition necessary for launching the plan (it must be consistent with the beliefs).
- **Body** — the actions to be performed.

The body of a plan is *not* merely a linear sequence of actions — it may contain further *goals*: "achieve f", "achieve f or g", "keep achieving f until g occurs". This gives rise to the hierarchical decomposition of goals into subgoals.

The run of the interpreter:

1. Initialisation: the initial beliefs B_0 and a top-level goal.
2. The intention stack contains the current goals in various stages of elaboration.
3. The interpreter takes the intention from the top of the stack and searches the plan library for plans with a matching goal.
4. Among the plans found it selects those whose *context* is consistent with the current beliefs — these constitute the current *options* / *desires*.
5. **Deliberation** = selecting one intention from the options:
   - The original PRS used *meta-plans* — plans about plans, which modified the agent's intentions. This turned out to be too complicated.

- A more practical variant: each plan carries a numerical *rating* (its expected utility), and the plan with the highest value is selected.

6. The selected plan is executed, which may lead to further intentions being pushed onto the stack.
7. If a plan fails, the agent selects another intention from the options and continues.

An illustration — Jason/AgentSpeak (an agent that brings its owner beer from the fridge):

```
available(beer,fridge).  limit(beer,10).    /* initial beliefs */
too_much(B) :- .count(consumed(_,_,_,B),Q) & limit(B,L) & Q > L.  /* rule */
+!has(owner,beer)                        /* triggering event */
  :  available(beer,fridge) & not too_much(beer)    /* context */
  <- !at(robot,fridge); open(fridge); get(beer); close(fridge);  /* body */
     !at(robot,owner); hand_in(beer).
+!at(robot,P) : not at(robot,P) <- move_towards(P); !at(robot,P).
```

The notation `+!g : c <- b` reads: "if the event of adding goal `g` occurs and the context `c` holds, execute the body `b`". The symbol `!` denotes an *achievement goal* (achieve), `+`/`-` the addition/removal of a belief or a goal, and `.send` and `.count` are built-in actions.

2.7 The reactive approach and its premises

During the 1980s and 1990s it became clear that minor adjustments to the symbolic approach were not enough, and alternative AI paradigms arose. Their common denominator is the **rejection of symbolic representation** and of reasoning based on syntactic manipulation. Three key theses:

Three principles of the reactive approach

Situatedness — intelligent behaviour depends on the environment in which the agent is *situated*; it cannot be studied in isolation.
Embodiment — intelligent behaviour is not just logic, it is the product of an agent that *has a body* and interacts physically with the world.
Emergence — intelligent behaviour *emerges* from the interaction of many simple behaviours; it is nowhere explicitly programmed.

Characteristics of reactive agents: they are *behavioural* (the emphasis is on developing and combining individual behaviours), *situated*, and *reactive* (the agent essentially only reacts to the environment, it does not deliberate). The representation is **subsymbolic**: connectionist networks, finite automata, simple IF–THEN rules.

2.8 Symbolic reactive planning: the subsumption architecture

Subsumption architecture

Rodney Brooks, 1986–1991; the most successful reactive approach. Brooks's three theses:

1. Intelligent behaviour can be generated *without symbolic representation.*
2. Intelligent behaviour can be generated *without explicit abstract reasoning.*
3. Intelligence is an *emergent property* of certain complex systems.

The intelligence of an agent is realised by a set of simple goal-oriented **behaviours**, for which the following holds:

- Each behaviour is itself an action selection mechanism (a *situation* → *action* pair).
- Each behaviour receives percepts and transforms them into actions, is responsible for one partial goal, and has the simple structure of a rule.
- Behaviours run *in parallel* and *compete* with one another for control of the agent.
- Behaviours are arranged into a **subsumption hierarchy**, which defines their priorities (a higher layer "subsumes" = suppresses a lower one).

The action selection mechanism in the subsumption architecture:

```
1: function ACTION(percept p)
2:     fired ← { b ∈ Behaviours | p satisfies the condition of b }      ▷ behaviours that "fire"
3:     for all b ∈ fired do
4:         if there is no b′ ∈ fired such that b′ ≺ b then              ▷ b is inhibited by nobody
5:             return the action belonging to behaviour b
6:         end if
7:     end for
8:     return null                                                     ▷ nothing fired, the agent does nothing
9: end function
```

Formally: a behaviour is a pair $\langle c, a \rangle$, where $c \subseteq Per$ is a condition and $a \in A$ an action; on the set of behaviours R an inhibition relation $\prec \subseteq R \times R$ (a strict total order) is defined. Mind the direction — in Wooldridge and in the lecture course, $b_1 \prec b_2$ means that b_1 **inhibits** b_2, that is, b_1 is *lower* in the subsumption hierarchy and takes *precedence*. The notation $b_1 \prec b_2 \prec \cdots \prec b_n$ thus lists behaviours from the highest priority to the lowest.

Properties: the mechanism is *simple* (although with dozens of behaviours the priorities tend to be tricky) and *very fast* — it can be implemented in hardware and has constant complexity.

Example: Steels's Mars explorer. The goal is to collect rare rock samples on a distant planet using a swarm of robotic explorers. The means: the base emits a navigation signal (the robots need to do nothing more than detect the *gradient* of the signal); each robot carries radioactive crumbs for *indirect communication* (stigmergy) with the others. Direct communication is not needed.

First iteration — a random walk and the return of a single robot (priority decreases from top to bottom):

R1	**if** I see an obstacle **then** change direction
R2	**if** I carry a sample **and** I am at the base **then** drop the sample
R3	**if** I carry a sample **and** I am not at the base **then** follow the gradient uphill
R4	**if** I see a sample **then** pick it up
R5	**if** true **then** move at random

Second iteration — better exploration by means of stigmergy. Samples often occur in clusters; a robot that has found one "tells" the others by dropping crumbs on its way home:

R6	**if** I carry a sample **and** I am at the base **then** drop the sample
R7	**if** I carry a sample **and** I am not at the base **then** drop 2 crumbs and follow the gradient uphill
R8	**if** I sense a crumb **then** pick up 1 crumb and follow the gradient downhill

Priorities (in the notation "the left one inhibits the right one", i.e. from the highest priority): R1 $\prec$ R6 $\prec$ R7 $\prec$ R4 $\prec$ R8 $\prec$ R5. A robot following a trail picks up one crumb at a time, so the trail gradually *evaporates* once the cluster has been exhausted. The swarm exhibits cooperative behaviour even though no robot has a model of the world or sends any messages.

2.9 Connectionist reactive planning: the agent network architecture

The subsumption architecture resolves conflicts by *fixed* priority, which is rigid. The connectionist approach replaces fixed priorities with **continuous activation spread through a network** — action selection is the result of the network's dynamics, not of a fixed ordering. This is "reactive planning": the agent behaves as if it were planning, but it never constructs an explicit plan.

Agent network architecture / *spreading activation networks*

Pattie Maes, 1989–1991. The agent is a set of **competence modules**, similar to Brooks's behaviours. Each module i has:

- **preconditions** c_i — what must hold for the module to be *executable*;

- **effects**: an *add list* a_i and a *delete list* d_i (as in STRIPS);
- an **activation level** α_i and a global **activation threshold** θ.

The modules are connected in a directed graph on the basis of their conditions — a match of pre- and postconditions forms edges, to which further edges representing temporal succession and conflicts are added. At each step activation is propagated; **the executable module with the highest activation is selected**, provided it exceeds the threshold θ.

Three sources and three paths of activation spreading:

- **From the state of the world**: modules whose preconditions are currently satisfied receive activation — "this can be done right now".
- **From goals**: modules whose add list contains a goal receive activation — "this leads to the goal".
- **From protected goals**: modules whose delete list contains an already satisfied goal are *inhibited* — "this would destroy what I already have".

Activation further spreads between modules along three types of link:

- **Successor links**: module x sends activation to module y if y adds something that x needs as a precondition — *forward* chaining ("once I am done, things can go on").
- **Predecessor links**: a module x that is not executable sends activation to modules that would satisfy its missing precondition — *backward* chaining ("help me so that I can run").
- **Conflictor links**: module x *inhibits* module y, which destroys its preconditions.

The behaviour of the network is governed by five global parameters: θ (the activation threshold, the only threshold in the system), ϕ (the activation injected into the network by the state of the world), γ (the activation injected by goals), δ (the activation removed by protected goals), and π (the *average* activation level to which the network is normalised after each step, so that the total energy does not change). The ratio γ/ϕ expresses precisely the **proactiveness vs. reactivity** trade-off; θ determines **deliberateness vs. speed** (a high threshold $\rightarrow$ the agent hesitates longer but decides better). If in a given step no executable module exceeds the threshold, θ is lowered by 10 % and the attempt is repeated — the network therefore cannot "freeze".

How the activation in one step is computed. The update proceeds in four phases: (1) *input from the environment* — each true fact distributes a quantum ϕ evenly among the modules that have it among their preconditions; (2) *input from goals* — each goal distributes γ among the modules that add it, and each protected goal *removes* δ from the modules that would destroy it; (3) *spreading between modules* — an executable module sends part of its activation forward to its successors (again divided evenly among the recipients), a non-executable module sends activation backward to modules that can satisfy its missing precondition, and conflictor links subtract the corresponding share of activation; (4) *normalisation* — the activations of all modules are rescaled so that their average is π (the total energy of the network remains constant). Then the executable module with activation above the threshold θ is selected; the module that has been executed resets its activation to zero.

A note on the lecture course: the slides present the activation threshold as a property of an individual module (and speak of it as a priority); in Maes's original model the threshold θ is *global* and the role of the "priority" is played by the current activation level α_i, which changes dynamically.

Comparison with subsumption: the ordering of behaviours is not fixed but arises *dynamically* from the current state of the world and the goals. The network is thus able to generate even multi-step "plan-like" sequences (activation chains backwards from the goal along predecessor links) without any explicit plan data structure ever being created — hence the name *reactive planning*. The price is sensitivity to the parameter settings and the impossibility of guaranteeing optimality or completeness.

2.10 Limitations of reactive architectures

- Reactive agents **have no model of the world**; everything must be derived from the current percepts. Sometimes they therefore have to *change the environment* in order to "remember" a piece of information (the radioactive crumbs) — externalised memory, stigmergy.
- They have only a **short-term view**: they act according to the current state and local information, and it is difficult to take global conditions and long-term goals into account.
- Emergence **is not a good engineering practice**: behaviours have to be tuned experimentally, and both a methodology and any possibility of verification are missing.
- The design is manageable for a few layers; **with dozens of behaviours it becomes intractable** (the number of interactions grows quadratically).

2.11 Hybrid architectures

Neither a purely reactive nor a purely deliberative architecture is ideal. Hybrid architectures combine both:

- A **deliberative / planning component** works at the symbolic level, creating representations and plans.
- A **reactive component** provides immediate responses without complex computation.

The components are usually arranged into **layers** (layered architectures), with the reactive layers as a rule taking *precedence* over the deliberative ones (safety before optimality).

Horizontal vs. vertical layering

Horizontal layering: all layers are connected *in parallel* to both the sensors and the effectors. Each layer is like a separate agent and proposes an action.

+ Conceptually simple: for n kinds of behaviour, n layers suffice.
− The layers influence one another and their proposals conflict; a **mediator function** is needed to resolve the conflict. It is the bottleneck: it has to consider m^n combinations (n layers, m possible proposals) and is a critical point of failure.

Vertical layering: the layers are connected *in series*.

- *One-pass*: a percept enters the lowest layer, passes upwards, and the action emerges from the topmost layer. A natural ordering and hierarchy of behaviours.
- *Two-pass*: percepts travel *bottom-up*, control commands (actions) *top-down*. The flow resembles management in a real company.

+ The need for mediation disappears, and the complexity of the interactions is linear ($n - 1$ interfaces).
− It is not robust: the failure of any one layer brings down the whole agent.

TouringMachines (Ferguson, 1992)

A **horizontal** three-layer architecture for a simulated agent in a traffic environment.

- The **modelling layer** — symbolic models of the agent, the environment, and the other agents; it predicts and resolves conflicts, sets goals, and passes them to the planning layer.
- The **planning layer** — proactive behaviour; it selects from pre-prepared plans (similarly to PRS) and assembles a route plan from them.
- The **reactive layer** — classical reactive *situation* → *action* rules, a fast immediate response (obstacle avoidance).

The layers work in parallel and their outputs are unified by a set of **control rules** — a concrete form of the mediator function; these can suppress the inputs and outputs of the layers (*censor* and *suppressor* rules).

InteRRaP (Müller, 1994)

A **vertical two-pass** three-layer architecture.

- The **cooperative planning layer** — social interaction, joint plans with other agents.
- The **local planning layer** — ordinary individual planning.
- The **behaviour-based layer** — reactive behaviour.

Each layer has *its own knowledge base* at the corresponding level of abstraction (knowledge about the world / plans / joint plans and models of the other agents). The layers communicate by two operations: *bottom-up activation* (a lower layer cannot handle the situation and passes it upwards) and *top-down execution* (a higher layer uses a lower one to carry out its decisions).

A real-world example: Stanley (a Volkswagen Touareg R5, Stanford). The autonomous vehicle that won the DARPA Grand Challenge 2005 (212 km across the Mojave Desert); a direct predecessor of today's autonomous vehicles. A layered architecture:

- the sensor layer → the abstract perception layer → the planning and control layer (the route plan and vehicle control) → the vehicle interface layer;
- plus a user interface layer (the panel, starting) and a global services layer (file system, communication, clock).

2.12 Complex architectures: IDA and the global workspace

The last family of architectures goes in the opposite direction to reactive minimalism: it attempts to integrate *everything* into a single agent — action selection, memory, deliberation, emotions.

IDA — *Intelligent Distribution Agent* (S. Franklin)

One of the most complex agent architectures ever built. It arose for a real task of the US Navy — assigning personnel to new billets (hence "distribution"); the architecture itself is distributed as well.

The starting point: global workspace theory (Baars, 1988–97) — one of the theories of consciousness:

- The mind *is* a multiagent system: it consists of many simple, mostly unconscious processes performing specialised operations.
- The processes communicate *rarely* and only through a shared *blackboard*-type memory, organised as an associative array (cf. chapter 5).
- The processes dynamically form higher-order **coalitions**.
- **The action selection mechanism:** the coalition that best matches the current inputs reaches "consciousness" (the global workspace) and is executed.
- There is a *hierarchy of contexts* representing the world at various levels — the context of goals, of percepts, of the senses, of concepts, and the cultural context.

Assessment: it combines many approaches from MAS and from the rest of AI (action selection, memory, deliberation, emotions), and that is precisely its weakness — many of the decisions are *ad hoc*, and tuning such a heterogeneous collection of models into an optimally cooperating whole is very difficult.

For comparing action selection mechanisms, IDA is useful as a *fourth* type of arbitration: subsumption decides by **fixed priority**, the Maes network by **activation level**, layered architectures by a **mediator function**, and IDA by the **competition of dynamically arising coalitions** for access to the global workspace.

2.13 Comparison of architectures

	Deductive	**BDI / PRS**	**Reactive**	**Hybrid**
Representation	symbolic (FOL)	symbolic (beliefs, plans)	subsymbolic / rules	mixed, per layer
Action selection	proving $Do(a)$	deliberation + plan from a library	priority / activation	layers + arbitration
World model	complete	partial, revisable	none	per layer
Speed	very low	medium	high (const.)	high in reflex
Reactivity	poor	medium (tunable)	excellent	good
Proactiveness	good	excellent	none	good
Design	difficult but formal	systematic	experimental	complicated

2.14 Exam summary

- **The deductive agent**: $DB \in 2^L$, $DB \vdash_\rho Do(a)$; two-phase action selection (a provable action $\rightarrow$ a consistent action $\rightarrow$ nothing). Elegant but impractical; the transduction problem and calculative rationality.
- **BDI**: two phases — deliberation (*brf*, *options*, *filter*) and means-ends reasoning (planning). Beliefs are subjective and fallible, desires may be inconsistent, intentions are commitments that persist and constrain further deliberation.
- **STRIPS**: the action descriptor $\langle P_a, D_a, A_a \rangle$, the planning problem $\langle B_0, O, G \rangle$, $B_i = (B_{i-1} \setminus D_{a_i}) \cup A_{a_i}$; an *admissible* plan ($B_{i-1} \models P_{a_i}$) vs. a *correct* one (moreover $B_n \models G$). Be able to work through the blocks world.
- **The BDI loop** and the role of *reconsider* and *sound*.
- **Commitment**: blind / single-minded / open-minded; bold vs. cautious agent; efficiency = achieved/all intentions; a slowly changing world $\rightarrow$ bold, a rapidly changing one $\rightarrow$ cautious.
- **PRS**: the first implementation of BDI; a plan = Goal + Context + Body; a plan library instead of planning; the intention stack; deliberation by meta-plans or by utility; descendants AgentSpeak/Jason, JAM, JACK, Jadex.
- The reactive approach rests on the triple *situatedness* — *embodiment* — *emergence*; it rejects symbolic representation.
- **Subsumption** (Brooks) = *symbolic* reactive planning: a set of simple *situation* $\rightarrow$ *action* behaviours running in parallel and ordered by fixed priorities. Be able to give Steels's Mars explorer and the role of stigmergy (the crumbs).
- **The agent network architecture** (Maes) = *connectionist* reactive planning: modules with preconditions and effects (add, delete), activation spreads along successor, predecessor and conflictor links, the sources of activation are the state of the world, the goals and inhibition from protected goals; the executable module with the highest activation above the threshold is selected. The parameters γ/ϕ tune proactiveness vs. reactivity.
- Limitations of reactive agents: no model of the world, a short-term view, a non-engineering design process, an unscalable number of layers.
- **Hybrid**: horizontal (parallel layers + a mediator, m^n combinations, a bottleneck) vs. vertical (serial, one-pass/two-pass, $n-1$ interfaces, fragile).
- **TouringMachines** = horizontal, 3 layers (modelling, planning, reactive) + control rules. **InteRRaP** = vertical two-pass, 3 layers (cooperative planning, local planning, behaviour-based) + its own KB per layer. **Stanley** as a real deployment.
- **IDA** (Franklin) + *global workspace theory* (Baars): the mind as a MAS of simple unconscious processes communicating through a blackboard; coalitions of processes compete for entry into "consciousness" and the winner is executed; a hierarchy of contexts. The most complex architecture, with many ad hoc decisions.
- Four types of arbitration between behaviours: **fixed priority** (subsumption), **activation level** (Maes), **a mediator function** (layered), **competition of coalitions** (IDA).

3 The pathfinding problem

[♦ beyond the slides: the whole chapter] The NAIL106 lecture lists planning and robotics among the topics it does *not* cover, and the slides contain nothing on pathfinding. The examination topic names it explicitly, however, so it has to be known. The chapter is therefore kept to the minimum that will do at the examination: A* with the properties of its heuristic, and then only what is specific to *multiagent* pathfinding — a dynamic environment and the coordination of several agents.

3.1 Formulation and context

Pathfinding is the most common concrete subtask a situated agent has to solve — whether as part of the planning layer of a hybrid architecture, as the action `move_towards(P)` in a BDI plan, or as a service provided to other agents. In a multiagent context two further specifics appear that the single-agent version does not know: the environment is **dynamic** (the map changes, obstacles appear) and the other agents are **moving obstacles** with which it is necessary to coordinate.

The pathfinding problem

We are given a weighted graph $G = (V, E)$ with non-negative edge weights $w : E \to \mathbb{R}_0^+$, an initial vertex $s \in V$ and a goal vertex $g \in V$ (or a set of goals G_{goal}). We seek a path $s = v_0, v_1, \ldots, v_k = g$ minimising $\sum_i w(v_{i-1}, v_i)$.
The graph is usually given *implicitly* by a successor function (a 4-/8-connected grid, a navigation mesh, a visibility graph, a grid of the robot's configuration space).

3.2 A*

Best-first search maintains for every vertex n the values

$g(n)$ = the cost of the best path $s \to n$ found so far,
$h(n)$ = a heuristic estimate of the cost $n \to g_{\text{goal}}$,
$f(n) = g(n) + h(n)$ — an estimate of the cost of the best path $s \to g_{\text{goal}}$ through n.

The A* algorithm (Hart, Nilsson, Raphael, 1968):

1: $OPEN \leftarrow \{s\}$ (a priority queue ordered by f), $CLOSED \leftarrow \emptyset$
2: $g(s) \leftarrow 0$, $f(s) \leftarrow h(s)$, all other $g \leftarrow \infty$
3: **while** $OPEN \neq \emptyset$ **do**
4: $n \leftarrow \arg\min_{x \in OPEN} f(x)$; remove n from $OPEN$
5: **if** n is a goal **then return** the reconstructed path
6: **end if**
7: add n to $CLOSED$
8: **for all** $m \in succ(n)$ **do**
9: $g_{\text{new}} \leftarrow g(n) + w(n, m)$
10: **if** $g_{\text{new}} < g(m)$ **then**
11: $g(m) \leftarrow g_{\text{new}}$; $parent(m) \leftarrow n$; $f(m) \leftarrow g(m) + h(m)$
12: insert/update m in $OPEN$ (and remove it from $CLOSED$ if it was there)
13: **end if**
14: **end for**
15: **end while**
16: **return** "no path exists"

Admissibility and consistency of a heuristic

A heuristic h is **admissible** if it never overestimates: $h(n) \leq h^*(n)$ for all n, where $h^*(n)$ is the true cost of the cheapest path from n to the goal.

A heuristic is **consistent** (monotone) if for every edge (n, m) the triangle inequality holds:

$$h(n) \leq w(n, m) + h(m), \qquad h(g_{\text{goal}}) = 0.$$

Properties: consistency ⇒ admissibility. With an admissible heuristic, A* is *optimal* (it finds the cheapest path). With a consistent heuristic, f is moreover

Special cases and variants: for $h \equiv 0$, A* degenerates into **Dijkstra's algorithm**. Typical heuristics on a grid are Manhattan $|\Delta x| + |\Delta y|$ (4 directions), octile (8 directions) and Euclidean. **Weighted A*** uses $f = g + \varepsilon h$, $\varepsilon > 1$ — it is faster, but the path found is at most ε times more expensive; **IDA*** saves memory ($O(d)$ instead of exponential). **A small example.** A 3×3 grid, movement in 4 directions, edge cost 1, start $(0, 0)$, goal $(2, 2)$, an obstacle at $(1, 1)$; the Manhattan heuristic $h(x, y) = |2-x|+|2-y|$ is both admissible and consistent. The start has $f = 0+4 = 4$; its successors $(1, 0)$ and $(0, 1)$ have $g = 1$, $h = 3$, $f = 4$; then $(2, 0)$ and $(0, 2)$ with $g = 2, h = 2, f = 4$, then $(2, 1)$ and $(1, 2)$ with $g = 3, h = 1, f = 4$, and finally the goal $(2, 2)$ with $g = 4, h = 0, f = 4$. All vertices on the optimal path have $f = 4$ — a typical manifestation of a consistent heuristic that estimates the remainder of the path exactly.

3.3 A dynamic environment: D* Lite

An agent moves and discovers along the way that the map is different from what it thought (a new obstacle, a changed cost). Replanning with A* from scratch at every step is expensive; the solution is **incremental** algorithms, which upon a local change recompute only the affected part.

LPA*, D* and D* Lite

LPA* (*Lifelong Planning A**, Koenig, Likhachev, 2001) — an incremental version of A* for a *fixed* start and goal. In addition to $g(n)$ it maintains for each vertex the value $rhs(n)$ (*right-hand side*), a *lookahead* value derived from the g values of its predecessors:

$$rhs(n) = \begin{cases} 0 & n = s, \\ \min_{p \in pred(n)} \big(g(p) + w(p, n)\big) & \text{otherwise.} \end{cases}$$

A vertex is **locally consistent** if $g(n) = rhs(n)$. If the cost of an edge changes, some vertices become inconsistent, are placed into a priority queue, and the algorithm propagates the change only where it is needed.

D* (*Dynamic A**, Stentz, 1994) — the original algorithm for a robot that moves and discovers obstacles. It searches *from the goal towards the start* (so that the values remain usable when the robot moves).

D* Lite (Koenig, Likhachev, 2002) — LPA* searched from the goal, with a correction of the priority queue as the robot moves. Functionally equivalent to D*, but substantially simpler; the *de facto* standard in mobile robotics. Orders of magnitude faster than repeated calls to A* — typically only a small neighbourhood of the change is recomputed.

An alternative: planning with free space. In an unknown environment the *freespace assumption* is often used: unknown cells are considered traversable, the agent follows the planned path, and as soon as it hits something it replans. This is exactly the scenario for which D* was designed.

3.4 Multi-agent path finding (MAPF)

[♦ beyond the slides: not in the slides, but it is the only genuinely multiagent part of the topic]

Multi-Agent Path Finding, MAPF

Input: a graph $G = (V, E)$ and k agents, where agent i has a start s_i and a goal g_i. Time is discrete; at each step every agent either traverses an edge or waits in place.
Conflicts — a solution has to be *conflict-free*: a *vertex conflict* (two agents are in the same vertex at the same time) and an *edge / "swap" conflict* (two agents exchange their positions in a single step).
The objective function: *sum-of-costs* $\sum_i cost_i$, or *makespan* $\max_i cost_i$. Finding an *optimal* solution is NP-hard.

Prioritized planning (the decoupled approach). The agents are ordered by priority and for $i = 1, \ldots, k$ the path of agent i is found by A* in *space–time* (a state is a pair ⟨vertex, time⟩) so as to avoid the trajectories already planned, which are stored in a **reservation table**. This variant is called *Cooperative A** (Silver, 2005). It is fast and it scales, but it is **incomplete** — there are solvable instances that no ordering of priorities solves (two agents in a narrow corridor where one of them has to pull into an alcove).

Conflict-Based Search (CBS)

Sharon et al., 2012–2015. A two-level *optimal* and *complete* algorithm that avoids the exponential joint state space V^k.
The low level: for a single agent an optimal path in space–time is found by A* while respecting a set of *constraints* of the form $\langle a_i, v, t\rangle$ ("agent i must not be in vertex v at time t").
The high level: it searches a *constraint tree* by cost. The root has an empty set of constraints and individually optimal paths. The node with the lowest cost is taken; if its paths are conflict-free, the solution is optimal. Otherwise the first conflict is chosen and the node *branches* into two children — into each a constraint is added for one of the colliding agents, and *only* that agent is replanned.

3.5 Exam summary

- A*: $f = g + h$; *admissibility* $h \le h^* \Rightarrow$ optimality; *consistency* $h(n) \le w(n, m) + h(m) \Rightarrow$ every vertex is expanded once. $h \equiv 0$ gives Dijkstra; weighted A* is ε-suboptimal.
- A dynamic environment: LPA* (the values g and *rhs*, local consistency $g = rhs$), D* Lite (search from the goal, incremental repair), the freespace assumption.
- MAPF: vertex and edge (swap) conflicts, sum-of-costs vs. makespan; optimal MAPF is NP-hard.
- Prioritized planning / Cooperative A* with a reservation table — fast but incomplete. **CBS** — the low level is A* with constraints, the high level branches the constraint tree on a conflict; optimal and complete.

4 Communication and knowledge in MAS: ontologies, speech acts, FIPA-ACL, protocols

4.1 From an agent to a multiagent system

We now know how to design a single agent. For agents to form a system they have to communicate with one another, and for that they need to resolve:

- **knowledge representation** — in what terms knowledge is expressed;
- **a shared vocabulary** — so that the same word means the same thing (this is the task of *ontologies*);
- **standard messages** — the syntax and semantics of communication (*ACL*);
- **predictable behaviour during communication** — *interaction protocols*;
- **technical problems** — distribution, agent mobility, asynchrony, unreliable communication channels.

4.2 What an ontology is

Ontology

Philosophically (from the Greek *to on* = that which is, *logos* = word): the study of being, according to Aristotle "first philosophy", a part of metaphysics.
In computer science: an **explicit and formalised description of a part of reality**. Usually a formal declarative description containing a *glossary* (definitions of concepts) and a *thesaurus* (definitions of relations between concepts). An ontology is a kind of vocabulary for storing and exchanging knowledge about a domain in a standard way.
Gruber (1995): "An ontology is a description (like a formal specification of a program) of the concepts and relationships that can formally exist for an agent or a community of agents." The frequently cited shortened version: *an ontology is an explicit specification of a conceptualisation.*

A motivating example (a dialogue between two people — or two agents). "Have you heard 7777? — No, what is it? — It's the new CD by the band SRPR, alt.rock with electronics and great lyrics. — I see." For them to understand each other, they have to share the following: 7777 is an album; it falls into the genres alt.rock and electronica; it has tracks, a performer (SRPR), and authors of the music and the lyrics; SRPR is a band, it has members, and they play instruments...

The formalism of an ontology

- **Classes** (concepts) — things that have something in common.
- **Individuals** (instances, objects) — concrete members of classes.
- **Properties** — attributes of classes and individuals.
- **Relations between classes**, in particular the **subclass** (*is-a*) relation — a *transitive* relation.
- Further relations and properties (basic knowledge about the domain).

The **structural part** (classes, relations, axioms) is usually called the *ontology* proper; together with **facts about concrete individuals** it forms a **knowledge base**.

An ontology of ontologies by strength of formalisation (from weak to strong): a vocabulary (a controlled vocabulary of selected terms) → a glossary (definitions of meanings in natural language) → a thesaurus (synonyms) → an informal hierarchy (Amazon, wikis) → a formal *is-a* hierarchy (class subsumption) → classes with properties → value restrictions ("every person has exactly one mother") → arbitrary logical constraints (which leads to complex inference algorithms).

An ontology of ontologies by scope:

- **Application** ontologies — the most common in practice, hard to reuse.
- **Domain** ontologies — a popular research subject in the semantic web.
- **Upper ontologies** — these are supposed to describe "everything" (Thing, Living thing, ...) and serve as a foundation for domain ontologies: BFO, GFO, UFO; WordNet and IDEAS are not suitable for formal inference. There are methodological objections against the very effort to build a universal ontology (Wittgenstein, *Tractatus logico-philosophicus*).

4.3 Ontology languages

Ontology languages are formal declarative languages for knowledge representation; they contain facts and inference rules and most often rest on first-order logic or on *description logic*.

Frames — a historical predecessor (Minsky), popular in classical AI and expert systems. They correspond to object-oriented programming:

Frame terminology	Object-oriented terminology
Frame	class of objects
Slot	property / attribute of an object
Trigger, accessor, mutator	method

XML was not created for ontologies, but it is used for them: its main advantage is the possibility of defining one's own tags, which naturally represent a vocabulary:

```
<product type="CD">
  <title>7777</title>
  <artist>SRPR</artist>
</product>
```

What it lacks, however, is a defined semantics — it describes only the structure of a document, not its meaning.

RDF — *Resource Description Framework*

The standard tool for knowledge representation for (but not only for) the web. It is **simple**: not very expressive, but with fast inference algorithms.
Everything is expressed by **triples** *subject – predicate – object*, that is, $Predicate(Subject, Object)$. For example, $MarriedTo(Karel, Jája)$, $FatherOf(Karel, Péťa)$. A set of triples forms a directed labelled graph (an *RDF graph*); resources are identified by IRIs. Serialisations: RDF/XML, Turtle, N-Triples, JSON-LD. The query language is SPARQL. The extension **RDFS** adds classes, `subClassOf`, `domain`, `range`.

OWL — *Web Ontology Language*

It too originates from the (semantic) web, and exists in versions 1 and 2. It is a *collection of several formalisms* for describing ontologies; the syntax may be XML, RDF, functional notation, Manchester syntax, or Turtle. An attempt at a formal and yet practically usable language.
Its basis is **description logic** — a *decidable fragment* of first-order logic.
The open world assumption (OWA): what we do not know is *undefined* — as opposed to the closed world assumption (databases, Prolog), where everything we do not know is *false*.

Description logic

Three kinds of entity: **concepts** (classes), **roles** (properties, predicates), and **individuals** (objects). An **axiom** is a logical expression about roles and concepts — unlike frames and object-oriented programming, which describe a class *completely*, axioms describe a class only *partially* (and they may come from several sources).
It is a whole family of logical systems according to the constructors allowed. The basic $\mathcal{ALC}$: negation, conjunction and disjunction of concepts, and restricted existential and universal quantifiers. Extensions: role hierarchies ($\mathcal{H}$), transitivity ($\mathcal{S}$), inverse roles ($\mathcal{I}$), cardinality restrictions ($\mathcal{N}$, $\mathcal{Q}$), nominals ($\mathcal{O}$).
A knowledge base is divided into the **T-Box** (the terminology — axioms about concepts and roles) and the **A-Box** (the assertional part — facts about individuals).

OWL profiles. *OWL 1 Lite* ($\mathcal{SHIF}$) — the simplest, close to RDF, with many restrictions for the sake of fast machine processing; *OWL 1 DL* ($\mathcal{SHOIN}$) — corresponds to description logic (it makes it possible to express, e.g., the disjointness of classes), is complete and decidable; *OWL 1 Full* — the greatest expressive power, but many problems are undecidable. *OWL 2* ($\mathcal{SROIQ}$) adds profiles: *EL* (inference in polynomial time, large medical ontologies such as SNOMED), *QL* (queries over a knowledge base, connection to relational databases), and *RL* (axioms in the form of rules).

KIF — *Knowledge Interchange Format*

Knowledge representation in first-order logic, with a LISP-like prefix syntax. It was designed within the DARPA *Knowledge Sharing Effort* project as the **inner language** of messages (the content), whereas KQML was the *outer* language (the envelope). Examples:

```
(salary 015-46-3946 widgets 72000)
(> (* (width chip1) (length chip1)) (* (width chip2) (length chip2)))
(forall (?F ?T) (=> (and (instance ?F Farmer) (instance ?T Tractor)) (likes
?F ?T)))
(exists (?F ?T) (and (instance ?F Farmer) (instance ?T Tractor) (likes ?F
?T)))
```

DAML+OIL — DARPA Agent Markup Language + Ontology Interchange Language, the direct predecessor of OWL, abandoned in 2006.

4.4 Reasoning about knowledge: epistemic logic

[♦ beyond the slides: the slides explicitly leave out modal logics; the topic does speak of "knowledge in MAS", though, so a minimum is worth knowing]

Epistemic logic and common knowledge

An agent's knowledge is modelled by the modal operator $K_i\varphi$ ("agent i *knows* that φ") with **Kripke possible-worlds semantics**: $K_i\varphi$ holds in a world w if and only if φ holds in *all* worlds that agent i cannot tell apart from w. **Knowledge** is axiomatised by the system **S5**; the key axiom is **T**: $K_i\varphi \Rightarrow \varphi$ (what I know is true). **Belief** (the operator B_i, cf. BDI) is axiomatised by **KD45**, where axiom T is replaced by the weaker **D**: $\neg B_i \bot$ — beliefs must be consistent, but they *may be false*. The difference between knowledge and belief is thus formally exactly axiom T.
For a group G: $E_G\varphi$ — everyone in G knows φ; **common knowledge** $C_G\varphi \equiv E_G\varphi \wedge E_G E_G\varphi \wedge \ldots$ (everyone knows that everyone knows, ...).
The coordinated attack (the two generals problem): two generals agree on a simultaneous attack through an unreliable messenger. No finite number of acknowledgements creates common knowledge — after k messages at most $E_G^k\varphi$ holds, never $C_G\varphi$. The lesson for MAS: *unreliable communication never creates common knowledge*, and hence never perfect coordination.

4.5 Why agent communication is different

In object-oriented programming, "communication" means calling a method of an object with parameters — the callee *must* comply. Agents cannot force another agent to do something, nor can they directly change its internal variables. They have to **perform a communicative act** with the aim of

- exchanging information, or
- *influencing* other agents to do something.

The other agents have their own goals and it is entirely up to them what they do with the information, request, or query they receive. This is the fundamental difference: **communication in MAS is an action, not a call**.

4.6 Speech act theory

Speech acts — Austin, 1962

Some parts of the use of language have the character of **actions**, because they change the state of the world in the same way as physical actions do — these are speech acts:

- "I now pronounce you man and wife."
- "I declare war."

This is a *pragmatic* theory of language: how speech is used to achieve goals. Typical performative verbs: *request*, *inform*, *promise*.

The three layers of a speech act

- The **locutionary act** — *what was said*; the utterance itself. "Make me some tea."
- The **illocutionary act** — *what was meant*; the locution + the performative meaning (a query, a request, ...). "She asked me for tea."
- The **perlocutionary act** — *what actually happened*; the effect of the speech act. "She got me to make her some tea."

For MAS the **illocutionary** part is the crucial one — that is the content of a message (the performative).

Searle elaborated the conditions under which a speech act is "felicitous". For the act *SPEAKER requests HEARER to perform an action*:

- **Normal input/output conditions**: the hearer can hear, it is not a scene in a film...
- **Preparatory conditions** (what must hold for the speaker to be able to choose this act): the hearer is able to perform the action; the speaker believes the hearer is able to perform it; it is not obvious that the hearer would perform it even without being asked.
- **Sincerity**: the speaker genuinely wants the action to be performed.

Searle's categories of speech acts

The older division: *request*, *advice*, *statement*, *promise*, ...
The newer (and standard) division into five classes:

- **Assertives** (representatives) — they inform the hearer (*inform*).
- **Directives** — they request the hearer to perform an action (*request*).
- **Commissives** — the speaker commits to something (*promise*).
- **Expressives** — the speaker expresses a mental state, an emotion ("thank you").
- **Declarations** — they change the state of affairs (war, marriage).

A speech act always has two components: a **performative verb** (request, query, inform) and a **propositional content** ("the window is closed").

The planning theory of speech acts (Cohen, Perrault, 1979)

"...modelling [speech acts] in a planning system as operators defined... in terms of the beliefs and goals of speakers and hearers. Speech acts are thus treated in exactly the same way as physical actions."
The semantics of speech acts is therefore given in the spirit of **STRIPS** (preconditions – delete – add), except that modal operators for *beliefs*, *abilities*, and *wants* are used.

Example: Request and Inform.

		Request(S,H,A)	Inform(S,H,φ)
cando precondition		(S believe (H cando A)) $\wedge$ (S believe (H believe (H cando A)))	(S believe φ)
want precondition		(S believe (S want requestInstance))	(S believe (S want informInstance))
Effect		(H believe (S believe (S want A)))	(H believe (S believe φ))

Note that the effect of *Inform* is *not* "the hearer comes to believe φ", but only "the hearer comes to believe that the speaker believes φ". An autonomous agent is not obliged to believe what anyone tells it — and that is precisely the difference from a method call.

4.7 KQML and FIPA-ACL

KQML — *Knowledge Query and Manipulation Language*

It arose in the 1990s within the DARPA *Knowledge Sharing Effort* (KSE) project together with KIF.

- **KQML** = the *outer* communication language ("the envelope of the letter"); it carries the *illocutionary* part of the message.
- **KIF** = the *inner* language; the propositional content of the message, knowledge representation.

A message is a list consisting of a *performative* and named parameters:

```
(ask-one
  :content (PRICE IBM ?price)
  :receiver stock-server
  :language LPROLOG
  :ontology NYSE-TICKS)
```

Parameters: content, force, reply-with, in-reply-to, sender, receiver, language, ontology.
Performatives: achieve, advertise, ask-about, ask-one, ask-all, ask-if, break, sorry, error, broadcast, forward, recruit-one, recruit-all, reply, subscribe, tell, ...

Criticism of KQML: a *commit* performative was missing, the semantics was not precisely defined, the set of performatives was unsystematic and needlessly large, and the transport mechanism was not standardised.

FIPA-ACL — *Agent Communication Language*

FIPA (the *Foundation for Intelligent Physical Agents*, today an IEEE standards committee) designed ACL as a **simplification and refinement of KQML**: a smaller and more systematic set of performatives and a *formally defined semantics*. A practical implementation: the JADE platform.

The structure of a message (13 fields — the performative and 12 parameters; only `performative` is mandatory):

Field	Meaning
`performative`	the type of the communicative act (illocutionary force) — *mandatory*
`sender`, `receiver`, `reply-to`	agent identifiers (AID)
`content`	the content of the message itself
`language`	the language of the content (SL, KIF, ...)
`encoding`, `ontology`	the encoding and ontology for interpreting the content
`protocol`	the interaction protocol the message is part of
`conversation-id`, `reply-with`, `in-reply-to`	matching messages into a conversation
`reply-by`	the deadline by which a reply is expected

```
(inform
  :sender agent1
```

```
        :receiver agent2
        :content (price good2 150)
        :language sl
        :ontology hpl-auction)
```

FIPA-ACL performatives (22 in total, the most important ones):

Group	Performatives
passing information	`inform`, `inform-if`, `inform-ref`, `confirm`, `disconfirm`
requesting information	`query-if`, `query-ref`, `subscribe`
requesting action	`request`, `request-when`, `request-whenever`, `cancel`
negotiation	`cfp` (*call for proposals*), `propose`, `accept-proposal`, `reject-proposal`
performing actions	`agree`, `refuse`, `failure`
errors, routing	`not-understood`, `proxy`, `propagate`

[♦ beyond the slides: the formal SL semantics is not in the slides, the performatives are]

The semantics of FIPA-ACL: FP and RE

The semantics is defined in the language SL (*Semantic Language*) — a modal logic with the operators $B_i\varphi$ (agent i believes φ), $U_i\varphi$ (it is uncertain), $I_i\varphi$ (it has the intention), Feasible, Done. For every performative the following are given:

- **FP** (the *feasibility precondition*) — what must hold for the sender to be able to perform the act;
- **RE** (the *rational effect*) — the effect for the sake of which it performs the act. The recipient is *not obliged* to bring about the RE (autonomy!) — the RE is only the sender's reason.

For example, for $\langle i, \text{inform}(j,\varphi)\rangle$:

$$\text{FP: } B_i\varphi \wedge \neg B_i\big(Bif_j\varphi \vee Uif_j\varphi\big), \qquad \text{RE: } B_j\varphi.$$

That is: "I believe φ and I do not believe that j already knows about φ (that it would know whether it holds or not — *Bif* — nor that it would hold an uncertain opinion about it — *Uif*); I want j to come to believe φ." For $\langle i, \text{request}(j,\alpha)\rangle$: FP: $FP(\alpha)[i\backslash j] \wedge B_i\text{Agent}(j,\alpha) \wedge \neg B_i I_j\text{Done}(\alpha)$, RE: $\text{Done}(\alpha)$.

A practical problem: this semantics is unverifiable — it is impossible to check from the outside that an agent really believes what it claims (the *sincerity assumption*). This is a well-known weakness of the mentalistic semantics of ACL; the alternative is a *social* semantics based on public commitments.

4.8 Interaction protocols

A single message is not enough; agents hold **conversations** with a prescribed structure. An interaction protocol is a pattern of admissible sequences of messages; FIPA standardises about a dozen of them (the FIPA-IP specification, AUML notation).

Protocol	Course
FIPA-Request	$I \to P$: `request`. $P \to I$: `refuse` (end), or `agree`; after execution `inform-done` / `inform-result`, or possibly `failure`. The reply may also be `not-understood`.
FIPA-Query	$I \to P$: `query-if` / `query-ref`. P: `refuse` or `agree`, then `inform` with the result or `failure`.
FIPA-Contract-Net	see below: `cfp` $\to n\times$(`propose` / `refuse`) $\to$ `accept-proposal` to the winner + `reject-proposal` to the others $\to$ `inform-done` / `failure`.
FIPA-Iterated-Contract-Net	like CNET, but after receiving the bids the initiator may announce *another round* of `cfp` (negotiating better terms).
FIPA-Subscribe	I: `subscribe`; P sends `inform` every time the monitored value changes, until a `cancel` arrives.
FIPA-Brokering / Recruiting	I asks a *broker*, which finds suitable providers, delegates the request to them, and returns the results (or possibly lets them answer directly).
Auction protocols	FIPA-English-Auction, FIPA-Dutch-Auction (see section 5.11).

A note on design. A protocol determines which messages are legal in a given state of the conversation — thereby it introduces *predictability* into otherwise autonomous behaviour and makes it possible to implement an agent as a finite automaton over the states of the conversation (which is exactly what JADE does with classes such as `AchieveREInitiator`, `ContractNetResponder`, and the like).

4.9 Exam summary

- An ontology = an explicit, formalised description of a part of reality (a glossary + a thesaurus); Gruber's definition. The formalism: classes, instances, properties, relations (in particular the transitive *is-a*), facts. An ontology + facts = a knowledge base.
- The scale of formalisation from a dictionary to arbitrary logical constraints; the division into application, domain, and upper ontologies.
- RDF = *subject–predicate–object* triples, simple and fast; RDFS adds classes.
- OWL = description logic (a decidable fragment of FOL), the *open world assumption*, T-Box vs. A-Box; the profiles Lite/DL/Full ($\mathcal{SHIF}$, $\mathcal{SHOIN}$), OWL 2 = $\mathcal{SROIQ}$ + the profiles EL/QL/RL.
- KIF = FOL in LISP syntax, the content of a message; KQML = the envelope.
- Epistemic logic: $K_i\varphi$ + possible worlds; knowledge = S5 (axiom T), belief = KD45. Common knowledge $C_G\varphi$ does not arise from unreliable communication (the coordinated attack).
- Agent communication is an *action*, not a method call — the recipient is not obliged to comply.
- Austin: speech acts; locution / illocution / perlocution; Searle: the felicity conditions and five categories (assertives, directives, commissives, expressives, declarations); a performative verb + propositional content.
- Cohen & Perrault: the semantics of speech acts in the STRIPS style (pre / effect) with modal operators; be able to write Request(S,H,A) and Inform(S,H,φ) — the effect of Inform is *H believes that S believes* φ.
- KQML (the envelope) + KIF (the content); parameters and performatives; the criticism (no commit, unclear semantics).
- FIPA-ACL: the structure of a message (performative, sender/receiver, content, language, ontology, protocol, conversation-id, reply-by), 22 performatives, the semantics via FP and RE in the SL language; the problem of unverifiability.
- Protocols: FIPA-Request, FIPA-Query, Contract Net (+ the iterated one), Subscribe, Brokering, auction protocols.

5 Distributed problem solving, cooperation, game theory, and auctions

The longest sentence of the topic joins two different situations, and the chapter therefore has two halves. In the first (sections 5.1–5.6) the agents *share a goal* and the only question is how to divide the work — that is *distributed problem solving* and *cooperation*. In the second (sections 5.7–5.13) the agents pursue *their own interests*, so game theory is needed (Nash equilibria, Pareto efficiency) together with mechanisms robust against manipulation (resource allocation, auctions).

5.1 Coherence and coordination

Agents have different goals, they are autonomous, they operate over time, and they decide at run time; they therefore have to be capable of *dynamic coordination*. In doing so they share two things: **tasks** (task sharing) and **information** (result sharing).

Coherence and coordination

Coherence — how well the system functions *as a whole* (the quality of the solution, the efficiency of resource use, degradation under failures).
Coordination — how well the agents manage to *minimise the overhead* associated with synchronisation: avoiding needlessly duplicated work, conflicts over resources, and unnecessary communication.
A system may be coherent even with poor coordination (the agents do the work three times over, but the result is correct) — at the price of waste.

CDPS — *Cooperative Distributed Problem Solving*

Lesser et al., 1980s. The cooperation of individual agents in solving a problem that exceeds their individual capabilities (information, resources, computational capacity).
The key assumption: the agents *implicitly share a common goal* and have no conflicts — **benevolence**. The measure of success is the *performance of the system as a whole*; an agent helps the whole even when this is disadvantageous for itself. Benevolence *enormously simplifies* the design of the system.

	Characteristics	**Difference**
PPS (*parallel problem solving*)	Bond, Gasser, 1980s; the emphasis is on parallel solving, with homogeneous and simple processors	the agents are not autonomous, this is essentially a parallel algorithm
CDPS	heterogeneous agents with sophisticated capabilities, a shared goal, benevolence	conflicts are not resolved, because they do not arise
MAS	a society of agents with *their own* goals, which they *do not share*	it is necessary to address: why and how to cooperate, how to identify and resolve conflicts, how to negotiate and bargain

5.2 Task sharing and result sharing

The general CDPS procedure has three phases:

1. **Problem decomposition** — hierarchical, recursive. The questions are *how* to decompose and *who* performs the decomposition. An extreme variant is the ACTORS model: a new agent is created for every subproblem, down to the level of individual instructions.
2. **Solving the subproblems** — during this the agents typically share information and may need to synchronise their actions.
3. **Solution synthesis** — again hierarchical, the composition of partial results.

Task sharing requires an *agreement* among the agents (who does what) — the typical mechanism

is the Contract Net. **Result sharing** may be *proactive* (an agent sends a result it believes will be useful) or *reactive* (it sends it upon request).

The benefits of result sharing

- **Confidence** — independent solutions of the same problem can be compared; agreement increases confidence in the result.
- **Completeness** — the agents share their *local* views and thereby a more global picture of the problem arises.
- **Precision** — sharing may make the overall solution more precise.
- **Timeliness** — the obvious advantage of distribution: a reduction in time.

5.3 The Contract Net

Contract Net Protocol (CNET)

Smith and Davis, 1977–1980. The metaphor of **task sharing by means of contracts** following the model of a market: the agent in need is the *manager*, the others are *contractors*. Four phases:

1. **Recognition** — an agent finds that it has a problem it cannot handle on its own (it lacks the capability, the resources, or the information, or the solution would be worse).
2. **Announcement** — it sends out (typically by broadcast) an announcement of the task: a *task description* (which may even be executable), *constraints* (deadline, price, quality), and *meta-information* (who may apply).
3. **Bidding** — the recipients decide whether they have the interest and the capabilities, and send a bid (a *tender*) with a price / time / quality.
4. **Awarding and expediting** — the manager selects the winner and concludes a contract with it; the contractor performs the task and returns the result.

Properties: simple, decentralised, robust against failures; a contractor may itself become the manager of a subtask, which leads to *hierarchical cascades of subcontracts*. It distributes the load dynamically. It is the most frequently implemented coordination mechanism of all (in FIPA it is standardised as FIPA-Contract-Net; in JADE it is available as a ready-made behaviour).
Limits: broadcasting is expensive with many agents; it assumes benevolence (a contractor does not lie about its capabilities) — otherwise it is necessary to use auction mechanisms that are robust against manipulation (section 5.11).

5.4 Blackboard systems

Blackboard system (BBS)

The very first scheme for cooperative problem solving (HEARSAY-II, speech recognition, 1970s). Results are shared through a common data structure — the **blackboard**.
The metaphor: several experts sit around a blackboard and can both write on it and read from it. Tasks appear on the blackboard dynamically; as soon as one of the experts sees a task it knows how to solve, it writes a partial solution on the blackboard. This continues until the solution of the whole task appears on the blackboard.
The roles:

- The **blackboard** — shared memory; typically structured into several *levels of abstraction* (a hierarchy of goals and actions). The formalism for representing information is essential.
- The **experts** (*knowledge sources*) — agents solving a particular type of task; they react to the goals on the blackboard, take their goal from the blackboard, and once selected they perform an action. They have a list of capabilities and an efficiency.
- The **arbiter** (the *control component*) — it selects which expert may approach the blackboard. It may be purely reactive, or it may reason about plans and maximise expected utility. It is responsible for solving the problem at a higher level.

Mutual exclusion over the blackboard is necessary — the bottleneck of the system.

Pros and cons of BBS:

+ A simple mechanism for coordination, cooperation, and the sharing of both tasks and results.
+ The experts *need not know about each other* and yet they cooperate (*loose coupling*); adding an expert requires no change to the others.
+ Messages on the blackboard can be rewritten — delegating tasks, creating subtasks, changing experts.
– All agents must have access to the blackboard and share the same architecture; it tends to be "crowded" around the blackboard (distributed hash tables partly address this).

Example: BBWar. The blackboard is a hash table mapping required capabilities to tasks; "open missions" are published on the blackboard. The experts form a hierarchy: a *commander* agent looks for tasks of the type "conquer a city" (`ATTACK-CITY`) and converts them into several tasks "attack a position" (`ATTACK-LOCATION`); soldiers of various types then pick from the blackboard the missions they have the capabilities for.

Example: FELINE (Wooldridge, Jennings, 1990) — a distributed expert system from the time before KQML and ACL. Knowledge sharing and the distribution of subtasks; each agent is a rule-based system that maintains: *Skills* — "I can prove/refute the following…" and *Interests* — "I am interested in whether the following holds…". Communication is a triple (sender, recipient, content), where the content is a hypothesis + a speech act (request, response, inform).

5.5 Distributed constraints (DisCSP and DCOP)

[♦ beyond the slides: not in the slides; the topic does speak of "distributed problem solving", to which the formal branch belongs]

DisCSP and DCOP

Distributed CSP: the variables $x_1, \ldots, x_n$ with domains D_i are divided among the agents (typically one variable per agent) and the constraints link variables of *different* agents. An assignment satisfying all constraints is sought; no agent sees the whole problem and each communicates only with its neighbours in the constraint graph.
DCOP is the optimisation variant: the constraints are *soft*, given by cost functions $f_{ij}(x_i, x_j)$, and $\sum_{ij} f_{ij}$ — that is, social welfare — is maximised.
Algorithms: *ABT* (asynchronous backtracking, Yokoo) — the agents are ordered and exchange `ok?` messages (their assignment downwards) and `nogood` messages (back upwards on a conflict); further *ADOPT*, *DPOP* and *Max-Sum*. Applications: meeting scheduling, assigning tasks to sensors, frequency allocation.

5.6 Agent interaction in the environment

Agents can help and hinder one another even *without communication*, simply by affecting the environment:

- Robots can carry a brick only if they push from two sides simultaneously (a positive interaction, requiring synchronisation).
- Robots crowd at a door and cannot open it (a negative interaction, requiring coordination).

Agents can form communities, hierarchies, and hostilities. **It is necessary to know the types of interaction** in a specific MAS — otherwise effective control mechanisms cannot be designed. The simplest model case, which the following chapter deals with, is the interaction of two rational (selfish) agents in an environment resembling a game.

5.7 The selfish agent

Why should an agent be honest about its capabilities? Why should it complete an assigned task? If the system is homogeneous (designed entirely by us), benevolence is a good strategy. Usually, however, the system contains agents with different interests — a conflict between the common goal and the goals of individuals (e.g. air traffic control: everybody wants safety, but every airline wants to land first). Sometimes it is not even clear what the common interest is. In that case it is better to consider **self-interested** agents.

Self-interested agent

The set of possible outcomes $O = \{o_1, o_2, \dots\}$ is *common* to all agents, but the preferences are not: each agent i has its own **utility function** $u_i : O \to \mathbb{R}$, which orders the outcomes.
Strategic thinking: maximise your utility under the assumption that everybody else acts rationally as well. This does *not* maximise the common utility, but it is a *robust* strategy.
A note: money is not a good utility for a human being — the utility function of money is non-linear and differs for the rich and the poor.

5.8 Normal-form games

Two-player game in normal form

Agents i and j with action sets A_i, A_j. The outcome is determined by both: $e : A_i \times A_j \to O$.
An agent chooses a **strategy** s_i:

- **pure** — a deterministic choice of a single action;
- **mixed** — a probability distribution over pure strategies (a random choice).

A game is written down as a **payoff matrix**: rows = the strategies of player i, columns = the strategies of player j, and a cell contains the pair u_i/u_j.
In general: a game in normal form is a triple $\langle N, (A_i)_{i\in N}, (u_i)_{i\in N}\rangle$, where N is the set of players.

Dominance

A strategy s_i **strictly dominates** a strategy s_i' if it gives agent i a *strictly* better outcome against *all* strategies of the opponent; it **weakly dominates** it if it gives an outcome at least as good against all of them and a strictly better one against at least one (this is also how the lecture course defines dominance). A strategy is **dominant** if it dominates all the other strategies of agent i.
The difference is not academic: truthfulness in the Vickrey auction is only *weakly* dominant (section 5.11).
A rational agent plays a dominant strategy if one exists. *Iterated elimination of strictly dominated strategies* (IESDS) is the basic solution technique: nobody will ever play a dominated strategy, so it can be struck out of the matrix, which may render further strategies dominated. When eliminating *strictly* dominated strategies the order does not matter and the result is unique; with *weakly* dominated strategies the order **does** matter and some equilibria may be lost.

Nash equilibrium

A pair of strategies (s_1^*, s_2^*) is a **Nash equilibrium** if:

- when agent i plays strategy s_1^*, it is best for agent j to play s_2^*, and at the same time
- when agent j plays strategy s_2^*, it is best for agent i to play s_1^*.

That is, s_1^* and s_2^* are *mutual best responses*. In general, for n players: a profile $s^* = (s_1^*, \dots, s_n^*)$ is a NE if for every i and every strategy s_i of that player

$$u_i(s_i^*, s_{-i}^*) \geq u_i(s_i, s_{-i}^*),$$

where s_{-i} denotes the strategies of all the others. **No player has a unilateral reason to deviate.**

A naive search for equilibria for n agents and m strategies requires going through m^n profiles. This definition concerns *pure* strategies — not every game has a NE in pure strategies (rock–paper–scissors) and some games have several.

Theorem 5.1 (Nash's theorem, 1950)**.** *Every game with a finite number of players and a finite number of strategies has at least one Nash equilibrium* in mixed strategies.

Computational complexity. Finding a NE is a *total* search problem (a solution always exists, the only question is how to find it). Since 2006 we have known that it is **PPAD-complete** — very roughly speaking "somewhat less hopeless than NP-completeness" (NP-completeness does not even make sense for a problem that always has a solution). For three and more players this was proved by Daskalakis, Goldberg, and Papadimitriou; shortly afterwards Chen and Deng supplied the remaining and hardest case of *two* players.

Pareto efficiency and social welfare

A strategy profile is **Pareto optimal** (Pareto efficient) if there is no other profile that would improve the outcome of one agent without worsening the outcome of some other one. A non-Pareto-optimal solution can therefore be improved "for free".
The **social welfare** of a profile is the sum of the utilities of all agents: $sw(o) = \sum_{i \in N} u_i(o)$. Maximising social welfare makes sense in cooperative scenarios — when the agents are on the same team, have a single owner, or solve a single task; the more homogeneous the system, the better. The maximum of social welfare is always Pareto optimal, but not conversely.

5.9 Classical games

The Prisoner's dilemma

$i \backslash j$	C (cooperate)	D (defect)
C	3/3	0/4
D	4/0	1/1

C = *cooperate*, cooperate with one's accomplice; D = *defect*, inform on him and get a lower sentence. The canonical notation for the payoffs: R (*reward* for mutual cooperation), P (*punishment* for mutual defection), T (*temptation* — I defect against a cooperator), S (*sucker's payoff* — I am betrayed). The conditions of the prisoner's dilemma:

$$T > R > P > S \qquad \text{and often in addition} \qquad 2R > T + S.$$

- $R > P$: mutual cooperation is better than mutual defection.
- $T > R$ and $P > S$: *defection is a dominant strategy for both agents.*
- (D, D) is the **only Nash equilibrium**.
- (D, D) is at the same time the **only non-Pareto-optimal** outcome of the game.
- (C, C) maximises social welfare $(3 + 3 = 6)$.

The rationality of each individual therefore leads to the collectively worst outcome.

Consequences of the prisoner's dilemma:

- **The tragedy of the commons**: a shared resource (pasture, fisheries, the atmosphere) is depleted because every individual maximises their own benefit.
- The question of *what being rational actually means* — and whether people are rational (experimentally, people cooperate more often than the theory predicts).
- **The shadow of the future**: in the *iterated* prisoner's dilemma the situation changes.

The iterated prisoner's dilemma and Axelrod's tournaments

If the PD is played *a finite number of times and both players know it*, backward induction yields D in every round (in the last round there is no reason to cooperate, hence none in the next-to-last either, ...). If it is played *infinitely* (or with an unknown end, i.e. with a probability of continuing), cooperation becomes rational — the *folk theorem*: every payoff profile better than the minimax can be supported as an equilibrium of the repeated game.

In 1980 Robert Axelrod organised tournaments of strategies for the iterated PD. The winner (twice) was the simplest strategy submitted, **Tit For Tat** (TFT): *cooperate in the first round, then always repeat the opponent's last move.* From the results Axelrod derived four properties of successful strategies:

- being **nice** — do not defect first;
- being **retaliating** — respond to defection immediately;
- being **forgiving** — forgive an opponent who returns to cooperation;
- being **non-envious** — do not strive to beat a *particular* opponent; TFT never scored more points than its counterpart, and yet it won the tournament.

As a fifth property Axelrod lists **clarity**: a strategy must be legible to the opponent, otherwise no cooperation can be established with it. The weakness of TFT: under noise (a move executed incorrectly) two TFT players get stuck in mutual retaliation — hence the variants *Generous TFT* (which occasionally forgives) and *Tit for Two Tats*.

Coordination game

$i \backslash j$	Ballet	Wrestling
Ballet	1/2	0/0
Wrestling	0/0	2/1

Also known as the *Battle of the Sexes* (BoS) or *Bach or Stravinsky*. The payoffs favour *agreement* of strategies, but the players differ in which agreement they care about more.

- Nash equilibria in pure strategies: (Ballet, Ballet) and (Wrestling, Wrestling).
- Social welfare and Pareto optimality: again both of these pairs.
- **The mixed NE**: each player plays their *more preferred* option with probability 2/3 and the less preferred one with 1/3. The expected utility in the mixed equilibrium is only 2/3 — *less* than in either pure equilibrium.

Computing the mixed NE (the general recipe for 2×2 games): player j chooses the probability q of playing Ballet so that player i is *indifferent* between their actions:

$$u_i(\text{Ballet}) = 1 \cdot q + 0 \cdot (1-q), \qquad u_i(\text{Wrestling}) = 0 \cdot q + 2 \cdot (1-q).$$

The equality $q = 2 - 2q$ gives $q = 2/3$, i.e. player j plays Ballet (their preferred option) with probability 2/3. Symmetrically, player i plays Wrestling with probability 2/3. The expected utility of player i is then $1 \cdot \frac{2}{3} = \frac{2}{3}$. **The key rule: in a mixed NE each player chooses their probabilities so as to make the *opponent* indifferent.**

The anti-coordination game (*Chicken*, *Hawk–Dove*). The payoffs support the choice of *different* strategies instead: $u(\text{swerve}, \text{swerve}) = 5/5$, $u(\text{swerve}, \text{dare}) = 1/6$, $u(\text{dare}, \text{swerve}) = 6/1$, $u(\text{dare}, \text{dare}) = 0/0$. The pure NE are the two *asymmetric* pairs; the maximum of social welfare (both swerve, $5 + 5$) is, on the contrary, *not* an NE. The symmetric mixed NE is $p = 1/2$ with expected utility 3.

5.10 Social choice and voting

[♦ beyond the slides: the slides devote a whole chapter to it, but the official wording of the topic does not name it — included as a complement to preference aggregation]

Game theory asks how agents will behave given their preferences. *Social choice theory* asks the

opposite: how to *derive* a collective decision from individual preferences. A **social welfare function** aggregates preferences into a complete ordering $>_{sw}$, a **social choice function** picks a single winner.

Voting systems: *majority* (over 50 %); *plurality* (a point for each first place); *plurality with elimination* (IRV — repeatedly drop the weakest candidate and redistribute their votes); *sequential majority elections* (a bracket of pairwise contests — *the order of the contests influences the result*); *the Borda count* ($N-1$ points for a first place, ..., 0 for a last one — consensual rather than majority voting); *the Slater system* (an acyclic ordering minimising the number of reversed edges in the tournament graph — NP-hard).

The plurality winner need not be a majority winner: with $o_1 = 40$ %, $o_2 = 30$ %, $o_3 = 30$ %, o_1 wins although 60 % of the voters did not want them. Hence **tactical voting**.

The Condorcet paradox

There are situations in which *whichever outcome we choose, a majority of the voters will be unhappy with it.*
Example: $V_1 : A > B > C$, $V_2 : B > C > A$, $V_3 : C > A > B$. Pairwise contests: A beats B (2:1), B beats C (2:1), C beats A (2:1). **The collective preference is cyclic**, even though all the individual preferences are linear — there is no *Condorcet winner*. The motif is the same as the non-existence of a pure Nash equilibrium in rock–paper–scissors.

Criteria for voting systems

The Pareto property (unanimity): if *all* voters prefer X to Y, then $X >_{sw} Y$ should hold. *Satisfied by* majority and Borda, *not satisfied by* sequential majority.
The Condorcet winner condition: whoever beats all the others in pairwise comparison should win. *Satisfied by* sequential majority elections, *not satisfied by* plurality or Borda.
Independence of irrelevant alternatives (IIA): whether $X >_{sw} Y$ should depend only on the relative ordering of X and Y in the voters' preferences. *Satisfied by* practically no common system.
Unrestricted domain (universality) and **non-dictatorship**.

Theorem 5.2 (Arrow's impossibility theorem, 1951)**.** *For three or more candidates there is no preferential voting system that converts individual orderings into a complete and transitive social ordering while satisfying unrestricted domain, non-dictatorship, the Pareto property, and IIA.*

A simpler formulation: for more than two candidates the only procedure satisfying the Pareto property and IIA is a *dictatorship*. This is a *negative* result about the limits of collective decision making — it does not follow that the only workable system is a dictatorship, but rather that every real system has to sacrifice some criterion (most often IIA).

The Gibbard–Satterthwaite theorem: every surjective and non-dictatorial social choice function for three or more candidates is **manipulable**. For MAS this is essential: an agent is a computer and tactical voting poses it no moral problem — hence the search for mechanisms robust against manipulation, which is the subject of auctions.

5.11 Auctions as a mechanism of resource allocation

Auction

An auction is a **market institution** in which the messages from traders contain price information — an offer to buy at a given price (a *bid*), or an offer to sell at a given price (an *ask*) — and which gives priority to higher offers to buy and lower offers to sell.
In MAS an auction is above all a **mechanism for dividing scarce resources among agents**.
In the real world: eBay, Sotheby's, mining licences, frequency bands for mobile operators. *In computer science:* allocating processor time, bandwidth, computational tasks, advertising slots.

The efficiency of an auction: it allocates the resource to the agent that wants it most (that has the highest valuation). The seller wants to *maximise* the price, the buyer to *minimise* it; each buyer has its own utility function.

Classification of auctions

By the valuation of the item:

- *private value* — the value depends only on the buyer (a concert ticket);
- *common value* — the value is the same for everybody, they just do not know it (mining rights) — hence the *winner's curse*: the winner is the one who erred the most in the upward direction;
- *correlated value* — a combination.

By protocol: the winner pays the first / the second price; *open cry* vs. *sealed bid*; a single round vs. several rounds.

5.12 The four basic types of auction

Auction	Protocol	Rational strategy	Notes
English	open cry, *ascending* price, the first (highest) price is paid	*dominant*: keep bidding up by ε until the price reaches my valuation, then withdraw	The most common one (antiques, art, the internet — about 95 % of auctions). The typical case in which the buyer overpays.
Dutch	open cry, *descending* price, the first to accept pays the current price	*no* dominant strategy exists; wait until the price falls just below my valuation — but this is a gamble	Fast; perishable goods (flowers, fish). Popular with sellers. It makes it impossible to estimate either the market price or the preferences of the others.
FPSB (*first-price sealed bid*)	a single round, sealed envelopes, the highest bid wins and pays its own price	*no* dominant strategy exists; bid *less* than one's valuation (*bid shading*) — the optimum depends on an estimate of the others	Sales of real estate and government bonds. It does not force buyers to use their utility function and does not reveal the market price. *Strategically equivalent to the Dutch auction.*
Vickrey (*second-price sealed bid*)	a single round, sealed envelopes, the highest bid wins but pays the *second* highest	*dominant*: bid one's valuation **truthfully**	Theoretically the most popular one; Google AdWords, stamp auctions. *Equivalent to the English auction* (with private values).

Why truthfulness is a dominant strategy in the Vickrey auction

Let my valuation be v, let me bid b, and let the highest bid of the others be p (which I do not know). I win exactly when $b > p$, and then I pay p; my profit is $v - p$.

- **I bid $b > v$ (overstate):** compared with $b = v$ something changes only when $v < p < b$. Then I win, but I pay $p > v$ — a profit of $v - p < 0$. *This harms me.*
- **I bid $b < v$ (understate):** compared with $b = v$ something changes only when $b < p < v$. Then I lose, although I would have won with a profit of $v - p > 0$. *I forgo a profit.*
- In no case *can* I gain more by bidding $b \neq v$ than by bidding $b = v$.

Hence $b = v$ is a (weakly) dominant strategy — regardless of what the others do. The auction is **strategy-proof** (incentive compatible) and at the same time *efficient* (the bidder with the highest valuation wins).

Example — a comparison of revenues. Three buyers with valuations $A = 80$ CZK, $B = 60$ CZK, $C = 30$ CZK. In an ideal scenario (with no additional information and no cheating) the auctions turn out as follows:

Auction	Winner	Price
English	A	≈ 61 (bidding continues until B withdraws at 60)
Dutch	A	80 or slightly less (79) — A must not risk accepting too late
FPSB	A	80 (or $60 + \varepsilon$ if A knows the valuations of the others)
Vickrey	A	60 (the second highest bid)

In all cases the item is obtained by the agent with the highest valuation (the auctions are *efficient*), but they differ in the price and in the information the auction reveals.

Why does FPSB come out at 80 when one is supposed to bid below one's valuation? Because the scenario assumes *zero* information about the others: agent A does not know how high B will go, and any retreat from 80 risks losing. The degree of shading is a function of one's beliefs about the others — which is why FPSB (and the Dutch auction) **has no dominant strategy**. If A knows the valuations of the others, it bids $60 + \varepsilon$ and the seller's revenue drops to the level of the Vickrey auction (cf. the revenue equivalence theorem below). In the English and Vickrey auctions, by contrast, a dominant strategy does exist and does not depend on what the others do.

The revenue equivalence theorem

Vickrey (1961), Myerson, Riley–Samuelson. Under the assumptions: risk-neutral buyers, independent private values from the same continuous distribution, the item is obtained by the buyer with the highest valuation, and the buyer with the lowest possible valuation has zero expected profit — **all four basic auctions give the seller the same expected revenue.**
The differences in practice therefore stem from violations of the assumptions: risk aversion (in which case FPSB/Dutch are better), common values and the winner's curse (in which case the English auction is better because it reveals information), the possibility of collusion among buyers (the English auction is more vulnerable), and the cost of communication (sealed bid is cheaper).

5.13 Combinatorial auctions and VCG

Combinatorial auction

Several items are auctioned simultaneously and buyers submit bids on *subsets* (*bundles*). The motivation: the values of items are not additive — *complementarity* (a left and a right shoe, two adjacent frequency blocks: $v(\{x, y\}) > v(x) + v(y)$) and *substitution* (two tickets for the same flight: $v(\{x, y\}) < v(x) + v(y)$).
The winner determination problem (WDP): select a set of mutually disjoint bids that maximises the sum of the prices. This is a generalisation of set packing — an **NP-hard** problem (and moreover hard to approximate). It is solved with ILP solvers and branch and bound (CABOB, CPLEX).
The second problem is *representational*: there are $2^m - 1$ possible bids on subsets; compact languages are used (OR, XOR, OR-of-XOR).

5.14 Further types of auction and resource allocation

- An *all-pay auction* — everybody pays and the highest bid wins; a model of lobbying, bribes, sporting contests, and patent races.
- A *silent* auction — a written variant of the English one. The *Amsterdam* auction — it starts as an English one, and once two bidders remain it switches to a Dutch one with a doubled price. *Tsukiji* (the Tokyo fish market) — everybody bids at once, and ties are resolved by rock–paper–scissors.

- A *double auction* — bids are submitted by buyers and sellers alike and are matched; a model of a stock exchange.

5.15 Mechanism design, VCG, and negotiation

[♦ beyond the slides: not in the slides; the topic names "resource allocation", to which this belongs — combinatorial auctions are mentioned in the slides in a single line]

Mechanism design

Game theory asks *how agents will behave under given rules*; mechanism design is "inverse game theory": *which rules to choose so that rational selfish agents will of their own accord do what we want* — typically tell the truth.

A mechanism is a pair: a rule choosing the outcome $x(\hat{v})$ from the declared preferences and a payment rule $p_i(\hat{v})$; the utility is $u_i = v_i(x(\hat{v})) - p_i(\hat{v})$. Desirable properties: **incentive compatibility** (it pays to declare the truth; the strongest variant, *strategy-proofness*, makes truthfulness a dominant strategy), **individual rationality** ($u_i \geq 0$), **efficiency** (maximisation of social welfare), **budget balance** and **computational tractability**. Not all of them can be had at once.

The revelation principle: for every mechanism there is a *direct* mechanism in which the agents declare their preferences truthfully and the outcome is the same. This is why the theory revolves around Vickrey and VCG. Without payments it runs into the Gibbard–Satterthwaite theorem.

Combinatorial auctions and VCG

In a **combinatorial auction** several items are auctioned at once and bids are placed on *subsets*, because the values are not additive: *complementarity* ($v(\{x,y\}) > v(x) + v(y)$, two adjacent frequency blocks) and *substitution* ($v(\{x,y\}) < v(x) + v(y)$). **The winner determination problem** (WDP) — to select disjoint bids maximising the sum of prices — is **NP-hard**.

VCG (Vickrey–Clarke–Groves) generalises the Vickrey auction: the allocation x^* maximising $\sum_i \hat{v}_i(x)$ is chosen and agent i pays **the externality it imposes on the others**:

$$p_i = \max_x \sum_{j \neq i} \hat{v}_j(x) - \sum_{j \neq i} \hat{v}_j(x^*).$$

It is truthful, efficient, and individually rational; for a single item it reduces to the Vickrey auction. Drawbacks: the WDP has to be solved $n{+}1$ times, the seller's revenue may be small (even zero), and it is vulnerable to collusion.

Example: items $\{x, y\}$; agent 1: $\{x\} \to 5$, agent 2: $\{y\} \to 4$, agent 3: $\{x, y\} \to 8$. The optimal allocation gives $5 + 4 = 9 > 8$. The payment is $p_1 = 8 - 4 = 4$ (without agent 1 the others would have 8, with the chosen allocation they have 4), so the profit is $5 - 4 = 1$; symmetrically $p_2 = 8 - 5 = 3$. The seller's revenue is $7 < 8$.

Negotiation: the monotonic concession protocol and Zeuthen's strategy

An auction requires a central auctioneer; the alternative is direct **negotiation**. The *negotiation set* consists of the individually rational and Pareto-optimal deals.

The monotonic concession protocol: in each round both agents simultaneously propose a deal. It ends if one agent's proposal is at least as good for the other as its own; otherwise at least one of them must *concede*, or the negotiation fails.

Zeuthen's strategy answers *who* should concede — the one who is less willing to risk conflict:

$$\text{Risk}_i = \frac{u_i(\text{own proposal}) - u_i(\text{the other's proposal})}{u_i(\text{own proposal})}.$$

The agent with the *lower* Risk concedes, and only by just enough to reverse the ratio. A pair of Zeuthen strategies is in Nash equilibrium and the resulting deal maximises the product of utilities (the Nash bargaining solution).

5.16 Exam summary

- Coherence (the performance of the whole) vs. coordination (minimising synchronisation overhead).
- CDPS: benevolence + a shared goal; distinguish it from PPS (homogeneous processors, a parallel algorithm) and from MAS (own goals, conflicts, negotiation).
- The three phases of CDPS: decomposition — solving the subproblems — synthesis. Task sharing (agreement) vs. result sharing (proactive/reactive) and its four benefits: confidence, completeness, precision, timeliness.
- **CNET**: recognition — announcement — bidding — awarding/expediting; the manager and the contractor, hierarchical subcontracting; standardised as FIPA-Contract-Net.
- **BBS**: the blackboard (a hierarchy of abstraction, mutual exclusion), the experts (skills), the arbiter (choosing an expert); loose coupling between experts; the bottleneck of access to the blackboard.
- DisCSP / DCOP: variables and constraints divided among agents, nobody sees the whole problem; ABT (`ok?` / `nogood`); DCOP maximises social welfare.
- A selfish agent maximises *its own* expected utility assuming the others do the same; this does not maximise the common utility.
- Normal-form games, pure vs. mixed strategies, dominance and dominant strategies, the iterated elimination of dominated strategies.
- **Nash equilibrium** = mutual best responses, nobody has a reason to deviate unilaterally; the naive search is m^n; **Nash's theorem** (existence in mixed strategies); finding one is PPAD-complete.
- **Pareto optimality** (one agent cannot be made better off without making another worse off) vs. **social welfare** (the sum of utilities); the maximum of social welfare is Pareto-optimal, but not conversely.
- **The prisoner's dilemma**: $T > R > P > S$; D is dominant; (D, D) is the only NE and the only non-Pareto-optimal outcome; (C, C) maximises social welfare. The tragedy of the commons. The iterated PD, the shadow of the future, Axelrod and TFT (nice, retaliating, forgiving, non-envious, clear).
- Be able to compute a mixed NE for 2×2: choose the probabilities so that the *opponent* is indifferent.
- Social choice: plurality / IRV / sequential / Borda / Slater; the Condorcet paradox (a cyclic collective preference); the criteria Pareto, Condorcet winner, IIA, universality, non-dictatorship; **Arrow's theorem** (Pareto + IIA $\Rightarrow$ dictatorship); Gibbard–Satterthwaite (every system can be manipulated).
- An auction = a mechanism for allocating scarce resources; an efficient auction gives the resource to whoever values it most. The dimensions: private/common value, open cry/sealed bid, first/second price, the number of rounds.
- The four basic types and their strategies: the English auction (bid up in ε steps to your valuation) and Vickrey (bid truthfully) have a *dominant* strategy; the Dutch auction (wait for your valuation $-\varepsilon$) and FPSB (bid below your valuation) **do not**. Be able to give the **proof of truthfulness** for Vickrey.
- Equivalences: English $\leftrightarrow$ Vickrey, Dutch $\leftrightarrow$ FPSB. **The revenue equivalence theorem** and when it fails (risk aversion, common values, the winner's curse, collusion). Be able to do the example with valuations 80/60/30.
- Mechanism design (inverse game theory), the revelation principle; combinatorial auctions and the WDP (NP-hard); **VCG** = the allocation maximising social welfare + a payment equal to

the externality imposed.
- Negotiation: the monotonic concession protocol + Zeuthen's strategy (the agent with the lower risk concedes).

6 Methods for agent learning and reinforcement learning

[♦ beyond the slides: the whole chapter] The lecture explicitly lists agent learning among the topics it does *not* cover. The examination topic names it, however, so it has to be known; the chapter is therefore written to the extent that will do at the examination — an overview of learning methods, MDPs and Q-learning in detail, the rest only in outline.

6.1 Why and how agents learn

Intelligent agents in MAS should be *adaptive*. Learning is a change in an agent's behaviour on the basis of experience; there is a whole range of methods:

Methods for agent learning

- **By the learning signal** (the three classical paradigms of machine learning): *supervised learning* (the agent receives the correct answers — e.g. learning a model of the opponent from observed moves, classifying percepts), *unsupervised learning* (finding structure in observations — clustering situations, discovering roles), and **reinforcement learning** (only a scalar reward from the environment is available; the most natural for autonomous agents, see the rest of this chapter).
- **Imitation learning** (*learning from demonstration*): the agent learns a policy from trajectories demonstrated by an expert — either by direct copying (*behavioural cloning*), or by *inverse RL*, in which the reward function is reconstructed from the expert's behaviour.
- **Evolutionary methods**: a population of controllers (parameters of reactive behaviours, neural network weights, rules) is bred by a genetic algorithm according to the utility achieved — the typical way in which the reactive architectures of chapter 2 are "tuned experimentally"; this also includes *learning classifier systems* and neuroevolution.
- **By what the agent learns**: a model of the environment (the transition function), a model of the opponent (*opponent modelling*), a value function, the policy directly, or, in PRS, the ratings of plans.
- **Individual vs. social learning**: the agent learns only from its own experience, or also by imitating and sharing experience with the others (learning in a swarm, shared experience in MARL).

The most common and theoretically most developed approach to learning in MAS is **reinforcement learning** (RL) — a change of behaviour by trial and error on the basis of *rewards* from the environment. RL fits naturally into the formalism of chapter 1: the policy π is an action selection mechanism and the reward is an operationalisation of utility.

We distinguish:

- **Single-agent learning** — easier, and classical RL algorithms can be used (Q-learning).
- **Multiagent RL** (*MARL*) — the agents learn *simultaneously*; this is formalised by Markov games (Nash and Pareto optimality), and the state of the art is deep MARL (MADRL).

6.2 The Markov decision process

MDP — *Markov Decision Process*

An MDP is a quintuple $\langle S, A, T, R, \gamma \rangle$:

- S — the state space; A — the action space;
- $T : S \times A \times S \to [0, 1]$ — the transition function, $T(s, a, s') = P(s_{t+1} = s' \mid s_t = s, a_t = a)$;
- $R : S \times A \times S \to \mathbb{R}$ — the reward function (the immediate reward for a transition);
- $0 \leq \gamma < 1$ — the discount factor, a trade-off between immediate and future reward.

The **Markov property** holds: the next state and reward depend only on the current state and action, not on the whole history. (Compare this with the state transformer function $\tau : R^A \to 2^E$ from the abstract architecture, which *does* have a history — an MDP is its Markovian simplification enriched with probabilities and rewards.)

A **policy** $\pi : S \to A$ (or stochastically $\pi(a|s)$). The goal is to find an *optimal policy* π^* maximising the expected discounted sum of future rewards:

$$\mathbb{E}\Big[\sum_{t=0}^{\infty} \gamma^t R(s_t, a_t, s_{t+1})\Big].$$

The **value function** $V^\pi : S \to \mathbb{R}$ assigns to a state the expected utility if the agent follows the policy π; the **action-value function** $Q^\pi(s, a)$ does the same for a state–action pair.

The Bellman equations

For a given policy (the *Bellman equation for evaluation*):

$$V^\pi(s) = \sum_{s'} T(s, \pi(s), s')\big[R(s, \pi(s), s') + \gamma V^\pi(s')\big].$$

For the optimal policy (the *Bellman optimality equation*):

$$V^*(s) = \max_{a \in A} \sum_{s'} T(s, a, s')\big[R(s, a, s') + \gamma V^*(s')\big],$$

$$Q^*(s, a) = \sum_{s'} T(s, a, s')\big[R(s, a, s') + \gamma \max_{a'} Q^*(s', a')\big].$$

The optimal policy is obtained from Q^* as $\pi^*(s) = \arg\max_a Q^*(s, a)$.

Value iteration — if we know the complete description of the MDP:

1: initialise $V_0(s) \leftarrow 0$ for all s
2: **repeat**
3: **for all** $s \in S$ **do**
4: $V_{k+1}(s) \leftarrow \max_a \sum_{s'} T(s, a, s')\big[R(s, a, s') + \gamma V_k(s')\big]$
5: **end for**
6: **until** $\max_s |V_{k+1}(s) - V_k(s)| < \varepsilon$
7: **return** $\pi(s) = \arg\max_a \sum_{s'} T(s, a, s')[R(s, a, s') + \gamma V(s')]$

The Bellman operator is a contraction with coefficient γ, so the iteration converges to V^* geometrically fast. An alternative is **policy iteration**: alternately *evaluate* the current policy (solve a system of linear equations) and *improve* it (greedily according to V^π); it converges in finitely many steps, because there are finitely many policies.

6.3 Q-learning and SARSA

In reality we usually **do not know** either T or R. The agent therefore learns from experience by interacting with the environment in discrete steps — this is *model-free* RL.

Q-learning (Watkins, 1989)

The agent maintains a *table* $Q(s,a)$ — an estimate of the discounted sum of future rewards if action a is performed in state s. After every transition $s \xrightarrow{a} s'$ with reward r:

$$Q(s,a) \;\leftarrow\; Q(s,a) + \alpha\Big[\underbrace{r + \gamma \max_{a'} Q(s',a')}_{\text{the target (TD target)}} - Q(s,a)\Big], \qquad 0 \le \alpha \le 1 \text{ is the learning rate,}$$

equivalently $Q(s,a) \leftarrow (1-\alpha)Q(s,a) + \alpha\big(r + \gamma \max_{a'} Q(s',a')\big)$.
Off-policy: the target uses $\max_{a'}$, so it learns the value of the *optimal* policy regardless of the policy the agent actually acts by. Provided that every pair (s,a) is visited infinitely often and α decreases suitably, $Q \to Q^*$.

The exploration–exploitation dilemma; ε-greedy

The agent has to *exploit* what it already knows, but at the same time *explore* new actions. The standard solution:

$$\pi(s) = \begin{cases} \text{a random action from } A & \text{with probability } \varepsilon, \\ \arg\max_a Q(s,a) & \text{with probability } 1-\varepsilon. \end{cases}$$

ε is typically decreased over time (*annealing*). Alternatives: softmax/Boltzmann $\pi(a|s) \propto e^{Q(s,a)/\tau}$, optimistic initialisation of Q, UCB (upper confidence bounds).

A concrete Q-learning step. Let $\alpha = 0.5$, $\gamma = 0.9$. The agent is in state s_1, performs action a_1, receives reward $r = 10$, and moves to s_2. The current values: $Q(s_1,a_1) = 4$, $Q(s_2,a_1) = 6$, $Q(s_2,a_2) = 8$.

$$\begin{aligned} \max_{a'} Q(s_2,a') &= \max(6,\,8) = 8, \\ \text{TD target} &= r + \gamma \max_{a'} Q(s_2,a') = 10 + 0.9 \cdot 8 = 17.2, \\ \text{TD error} &= 17.2 - Q(s_1,a_1) = 17.2 - 4 = 13.2, \\ Q(s_1,a_1) &\leftarrow 4 + 0.5 \cdot 13.2 = \mathbf{10.6}. \end{aligned}$$

SARSA

State–Action–Reward–State–Action. The update uses the *actually chosen* next action a' (according to the same policy the agent acts by):

$$Q(s,a) \leftarrow Q(s,a) + \alpha\big[r + \gamma\, Q(s',a') - Q(s,a)\big].$$

On-policy: it learns the value of the policy the agent actually executes (including random exploration). The consequence: SARSA is *more cautious* — in the *cliff walking* task Q-learning learns the optimal path right at the edge of the precipice (and with ε-greedy it occasionally falls in), whereas SARSA learns a safer path further from the edge, because it factors in the risk of its own exploration.
A generalisation of both: TD(λ) with *eligibility traces*, which spread credit over several preceding steps.

6.4 Policy-based methods

Policy gradient and actor–critic

Instead of value functions one can optimise *the policy directly*. The policy is parameterised by a vector z, that is $\pi(a \mid s, z)$, and z^* is sought by gradient ascent on the expected return. **REINFORCE** (Williams, 1992) estimates the gradient from Monte Carlo simulations of whole episodes:

$$z_{t+1} = z_t + \alpha\, G_t\, \nabla_z \ln \pi(A_t \mid S_t, z_t),$$

where G_t is the return from time t — intuitively: increase the probability of actions that were followed by a high return. Advantages over value-based methods: it handles *continuous* action spaces and naturally produces *stochastic* policies (needed in games with mixed equilibria); the drawback is the high variance of the gradient estimate.

Actor–critic combines both approaches: the *actor* represents the policy, the *critic* learns the value function and reduces the variance. With the **advantage function** $A(s,a) = Q(s,a) - V(s)$ the gradient is scaled by the relative advantage of an action instead of by the return. Variants: A3C, A2C, PPO, SAC, DDPG.

Deep RL: DQN replaces the Q table by a neural network; the key tricks are *experience replay* (breaking the correlations between samples) and a *target network* (stabilising the target).

6.5 Multiagent reinforcement learning

Markov game (stochastic game)

A generalisation of the MDP to N agents: $\langle N, S, (A_i)_{i\in N}, T, (R_i)_{i\in N}, \gamma\rangle$. The **joint action space** is the product of the individual ones, $\mathbf{A} = A_1 \times \cdots \times A_N$; the transition depends on the actions of *all* the agents, and every agent has *its own* reward R_i and policy π_i, but its value function depends on the policies of all of them.

The link with game theory: an agent's policy is a *best response* to the joint policies of the others; the joint policies may be in a **Nash equilibrium** and may be **Pareto-optimal**. Special cases: $N = 1$ gives an MDP; a game with a single state gives a normal-form game; $N = 2$ with zero sum gives a *minimax* game.

Minimax-Q (Littman, 1994) — for two-agent zero-sum games; instead of $\max_{a'} Q(s', a')$ the value of the mixed minimax equilibrium of the matrix $Q(s', \cdot, \cdot)$ is used, which is a linear program.

Nash-Q does the same for general-sum games, but converges only under strong assumptions.

Why MARL is hard

- The agents update their policies **simultaneously**, so from the point of view of one agent the environment is **non-stationary** — the basic convergence assumption of RL is violated (a "moving target").
- The joint action space grows **exponentially** with the number of agents.
- The environment is only **partially observable** (a Dec-POMDP, whose optimal solution is NEXP-complete).
- **Credit assignment**: with a shared reward it is unclear which agent contributed.

The standard countermeasure: *centralised training with decentralised execution* (CTDE) — during training the critic sees the state and the actions of all agents, at run time the actor sees only its own local observations (MADDPG, COMA, QMIX); further, parameter sharing between homogeneous agents and *opponent modelling*.

Self-play. An agent learns by playing against itself (or against earlier versions of itself). In zero-sum games with perfect information self-play reliably leads to strong strategies: TD-Gammon, AlphaGo Zero, AlphaZero. The risk is *cyclic* preferences among strategies (rock–paper–scissors), where naive self-play oscillates — hence leagues of opponents and population-based methods.

6.6 Exam summary

- Methods of agent learning: supervised / unsupervised / reinforcement; imitation learning and inverse RL; evolutionary methods (LCS, neuroevolution); what the agent learns (a model of the environment, a model of the opponent, a policy); individual vs. social learning. RL is the most common.
- **MDP** $= \langle S, A, T, R, \gamma \rangle$, the Markov property, a policy π, the value functions V^π, Q^π; the Bellman optimality equation; value iteration (a contraction, it converges) and policy iteration.
- **Q-learning**: $Q(s,a) \leftarrow Q(s,a) + \alpha[r + \gamma \max_{a'} Q(s',a') - Q(s,a)]$, off-policy, tabular; be able to compute one step with numbers.
- **SARSA**: $\cdots + \alpha[r + \gamma Q(s',a') - Q(s,a)]$, on-policy, more cautious (cliff walking). ε-greedy as the answer to the exploration/exploitation dilemma.
- **Policy gradient** (REINFORCE, $\nabla \ln \pi$ times the return) and **actor-critic** (the actor = the policy, the critic = the value function, the advantage $A = Q - V$); DQN (replay buffer, target network).
- **Markov game** = an MDP for N agents with a joint action space and individual rewards; best response, Nash equilibrium, Pareto optimality. **Minimax-Q** (zero sum, an LP in every update), Nash-Q (general sum).
- The problems of MARL: non-stationarity, an exponential joint action space, partial observability, credit assignment; the remedy is CTDE. Self-play and AlphaZero.

7 Design methodologies, languages, and MAS platforms

7.1 Agent-oriented software engineering

Object-oriented methodologies (UML) are not sufficient, because agents are autonomous (their methods cannot be called directly), they have their own goals, and the relations between them are *organisational* (roles, protocols, commitments) rather than structural. This is why the methodologies of **AOSE** (agent-oriented software engineering) arose.

Gaia (Wooldridge, Jennings, Kinny, 2000)

One of the most cited AOSE methodologies. The key metaphor: a MAS is an **organisation** and agents play **roles**. It proceeds from requirements through analysis to design (leaving the implementation to the particular platform).

The analysis phase — two abstract models. The *roles model*: every role is described by four attributes — *permissions* (which resources it may read/modify), *responsibilities* (what it has to ensure; these split into *liveness* properties "something good happens", described by a regular expression over activities and protocols, and *safety* properties "nothing bad happens", described by invariants), *activities* (computations performed by the role itself), and *protocols* (interactions with other roles). The *interactions model*: the definition of the protocols between roles (purpose, initiator, responder, inputs, outputs).

The design phase — three concrete models: the *agent model* (the assignment of roles to agent types and the numbers of instances), the *services model* (the functions provided by a role — inputs, outputs, preconditions, effects), and the *acquaintance model* (a directed graph of who communicates with whom — a check for bottlenecks and excessive coupling).

Limitations: the original Gaia assumes a closed system with cooperative agents and a static organisation; the extensions *ROADMAP* and *Gaia v.2* relax these limitations.

[♦ beyond the slides: the slides name only Gaia and roles; the other methodologies are for orientation] **Prometheus** (Padgham, Winikoff, 2002) is a practical methodology aimed at **BDI agents**, with three phases: *system specification* (goals, *functionalities*, the interface with the outside world — *percepts* and *actions*), *architectural design* (agent types, protocols, messages) and *detailed design* (capabilities, plans, events — the building blocks of BDI). **Tropos** builds on the modelling of *actors and goals*

(the i^* framework) and is the only one to cover the early-requirements phase as well; it works with the notions *actor*, *goal* and *softgoal* (a non-functional requirement without a sharp satisfaction criterion), *plan*, *resource* and *dependency*. There are also **MaSE**, **INGENIAS**, **ADELFE** and **PASSI**.

7.2 The FIPA standard and the architecture of a platform

The FIPA reference architecture of an agent platform

- **AMS** (*Agent Management System*) — a mandatory component; the "registry" of the platform: it keeps records of agents, assigns them unique *AIDs* (agent identifiers), manages the life cycle (create, suspend, resume, terminate), and provides the *white pages*.
- **DF** (*Directory Facilitator*) — the *yellow pages*: agents register with it the *services* they offer and look up service providers.
- **MTS** (*Message Transport Service*) — the delivery of ACL messages within a platform and between platforms (the IIOP and HTTP protocols; bit-efficient, XML, and string encodings).
- **ACC** (*Agent Communication Channel*) — the interface to the MTS.

The standard further defines the ACL language, content languages (SL), interaction protocols, and ontology management.

7.3 Languages and platforms

[♦ beyond the slides: AGENT-0 and Concurrent MetateM are not in the slides; JADE and Jason are] The historical starting point is **agent-oriented programming** (Shoham 1993) — a paradigm in which the state of a program consists of *mental categories* (beliefs, commitments, capabilities) and the program is a set of *commitment rules*. The first language was **AGENT-0**; a different route is taken by **Concurrent MetateM**, where an agent is a *directly executable specification* in temporal logic. Both continue the line of deductive agents (section 2.1); the most successful agent language in practice today is AgentSpeak/Jason.

System	Type	Characteristics
JADE	Java, a FIPA platform	*Java Agent DEvelopment Framework*, one of the most widespread platforms for MAS, fully compliant with FIPA. An agent = a class inheriting from `Agent` with a `setup()` method; behaviour is composed of `Behaviour` objects (`OneShot`, `Cyclic`, `Ticker`, `FSMBehaviour`, `SequentialBehaviour`), which are scheduled by a cooperative (non-preemptive) scheduler inside the agent's single thread. Messages: `ACLMessage`, `send()`/`blockingReceive()`, `MessageTemplate`. Ready-made classes for protocols (`ContractNetInitiator`, `AchieveREResponder`), debugging tools (RMA, Sniffer, Introspector), containers, and agent mobility.
Jason	an AgentSpeak(L) interpreter in Java	The reference implementation of **AgentSpeak**, that is, of a **BDI** language (logic programming + BDI), a direct descendant of PRS. An agent program = initial *beliefs*, *rules*, and a *plan library* of the form `trigger : context <- body`. The trigger is an *event* (`+b` the addition of a belief, `+!g` the addition of a goal, `-!g` the failure of a goal). The interpreter cycle: perceive → update the beliefs → select an event → find the *relevant* plans (by trigger) → select the *applicable* ones (by context) → create an intention → execute a step. Together with `CArtAgO` (the environment) and `Moise` (the organisation) it forms *JaCaMo*.
NetLogo	an environment for ABM	A language derived from Logo for *agent-based modelling*. The entities: *turtles* (mobile agents), *patches* (cells of the environment), *links*, and the *observer*. A model library (Segregation, Wolf Sheep Predation, Ants, Flocking) and *BehaviorSpace* for parametric studies. Intended for emergent behaviour and for teaching, not for FIPA communication.
SPADE	Python, XMPP	*Smart Python Agent Development Environment*. Communication via standard **XMPP** instead of a proprietary MTS; an agent = a class with *behaviours* (`CyclicBehaviour`, `PeriodicBehaviour`, `FSMBehaviour`), asyncio, and a web interface; popular thanks to its connection to the Python machine learning ecosystem.

Further environments: *Jadex*, *JACK* and *JAM* (further implementations of BDI/PRS; Jadex builds a BDI layer on top of JADE); *MASON*, *Repast* and *GAMA* (simulation platforms for agent-based models); *PettingZoo*, *Gymnasium* and *OpenSpiel* (standard environments for multiagent reinforcement learning).

7.4 Exam summary

- Why OO methodologies do not suffice: autonomy, own goals, organisational (not structural) relationships.
- **Gaia**: organisation + roles; analysis = the roles model (permissions, responsibilities with liveness and safety, activities, protocols) + the interaction model; design = the agent, services, and acquaintance models. It assumes cooperative agents and a static organisation.
- **Prometheus** (aimed at BDI: functionalities → agent types → capabilities and plans) and **Tropos** (actors, goals and softgoals, dependencies; it also covers early requirements).
- **The FIPA platform**: AMS (white pages, life cycle, AID), DF (yellow pages, services), MTS/ACC (delivery of ACL messages).
- **JADE** = Java, FIPA, `Agent` + `Behaviour` + `ACLMessage`, ready-made protocols and debugging tools. **Jason/AgentSpeak** = a BDI language, a plan of the form `trigger : context <- body`, a descendant of PRS. **NetLogo** = ABM simulation (turtles/patches/links). **SPADE** = Python + XMPP.

In closing: how to approach the examination

The topic is broad; the examiner usually checks that you understand the *connections*. Four axes around which the questions revolve:

1. **Reactivity vs. proactiveness** — from the definition of an intelligent agent through commitment strategies (bold vs. cautious) and the parameters γ/ϕ in the Maes network to the layers of hybrid architectures.
2. **Individual vs. collective rationality** — a selfish agent leads to (D, D) in the prisoner's dilemma, to the tragedy of the commons, and to tactical voting; hence the need for mechanism design (Vickrey, VCG), that is, for designing *rules* under which it pays a selfish agent to be truthful.
3. **Symbolic vs. subsymbolic representation** — FOL, STRIPS, ontologies and ACL versus subsumption, activation networks, and neural networks in RL; practice is hybrid.
4. **Theory vs. practice** — elegant but non-constructive definitions (the optimal agent, Nash's theorem, Arrow's theorem) versus usable algorithms (A*, CBS, Q-learning, CNET, JADE). Do not forget the complexity: the PPAD-completeness of finding an NE, the NP-hardness of the WDP and of optimal MAPF.

Small points that are often asked about: the difference between a *desire* and an *intention*; why the effect of `inform` is *not* that the recipient believes; why (D, D) is the only *non*-Pareto-optimal outcome of the prisoner's dilemma; why truthfulness is dominant in the Vickrey auction; the difference between off-policy Q-learning and on-policy SARSA; and what CBS does at the high and at the low level.